MONGODB

SECTION 3

ENG. ASMAA FATHY

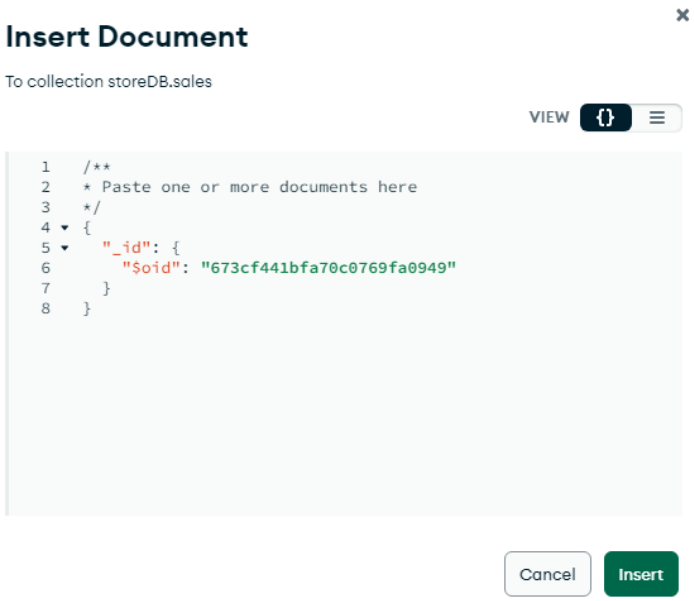
INSERT DOCUMENTS

- Compass provides two ways to insert documents into your collections: JSON Mode and a Field-by-Field Editor.
- JSON Mode : Allows you to write or paste JSON documents in the editor. Use this mode to insert multiple documents at once as an array.
- Field-by-Field Editor : Provides a more interactive experience to create documents, allowing you to select individual field values and types. This mode only supports inserting one document at a time.

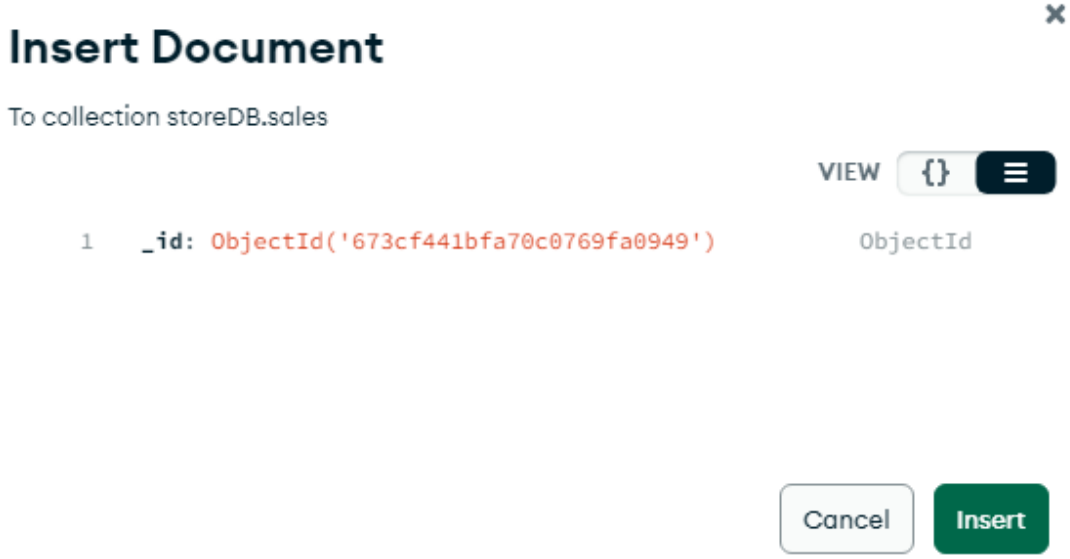
PROCEDURE TO INSERT DOCUMENTS INTO YOUR COLLECTION:

1. Click the Add Data dropdown and select Insert Document.

2. Select the appropriate view based on how you would like to insert documents.
   - Click the { } brackets for JSON view. This is the default view.
   - Click the list icon for Field-by-Field mode.

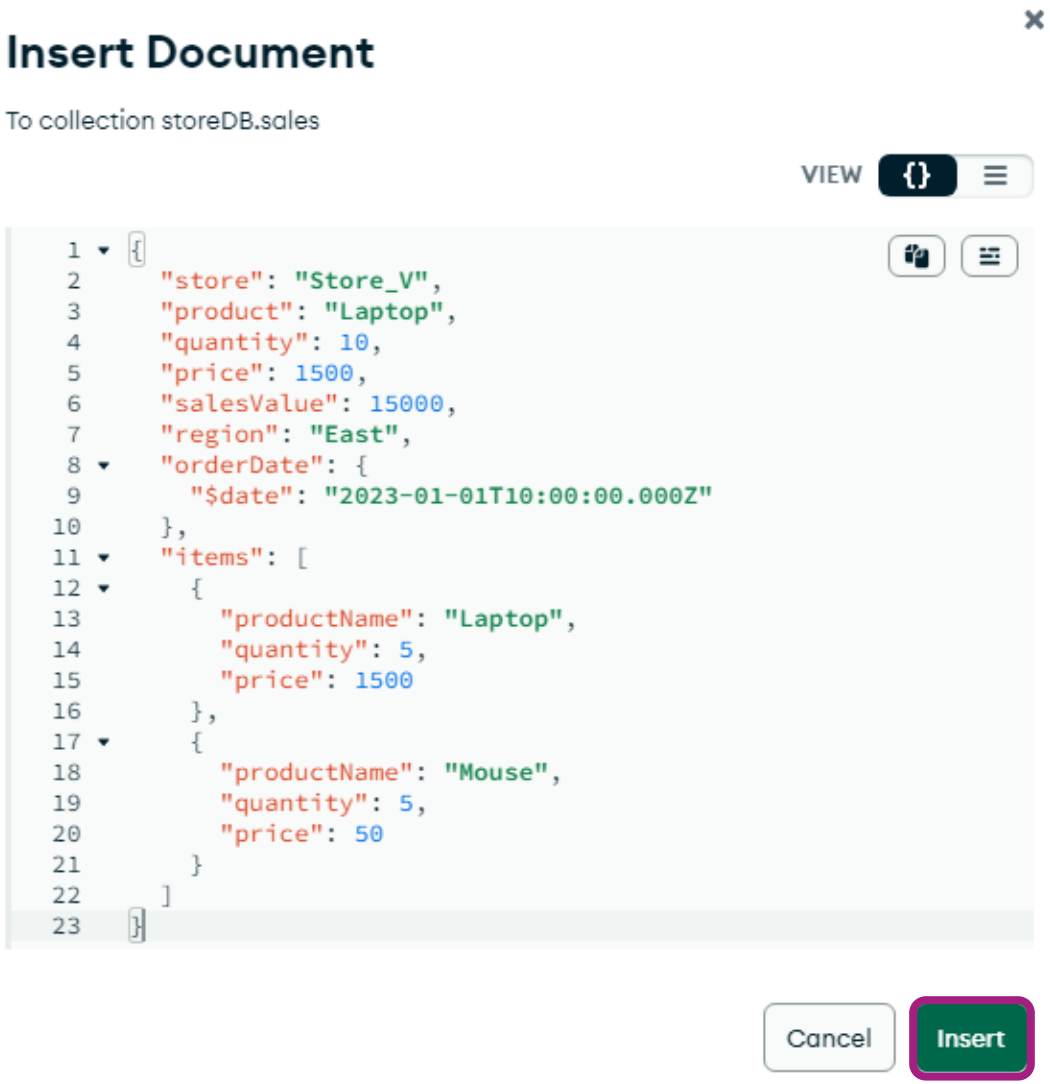


JSON view



Field-by-Field view

3. In JSON format, type or paste the document(s) you want to insert into the collection. To insert multiple documents, enter a comma-separated array of JSON documents.
   - Note : If you do not provide an ObjectId in your document, Compass automatically generates an ObjectId.
4. Click **Insert**.

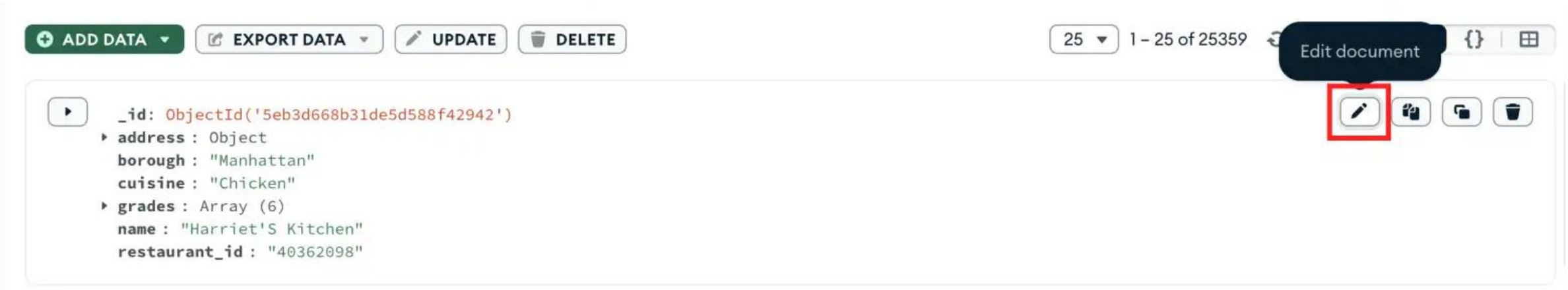


MODIFY SINGLE DOCUMENT

- You can edit existing documents in your collection.
- When you edit a document in List or Table view, Compass performs a findOneAndUpdate operation and updates only those fields that you have changed.
- When you edit a document in JSON view, Compass performs a findOneAndReplace operation and replaces the document.

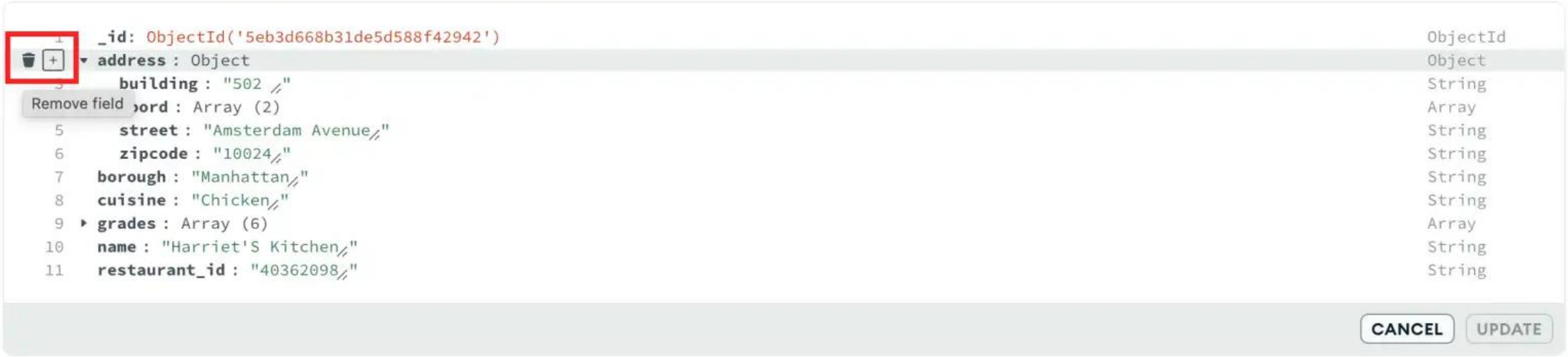
TO MODIFY A DOCUMENT, HOVER OVER THE DOCUMENT AND CLICK THE PENCIL ICON:

- After you click the pencil icon, the document enters edit mode. You can now make changes to the fields, values, or data types of values.



TO DELETE A FIELD FROM A DOCUMENT, CLICK THE TRASH ICON TO THE LEFT OF THE FIELD:

- Once selected, the field is marked for removal and appears highlighted in red. Compass asks for confirmation that you want to update the document by removing the field.



ADD NEW FIELDS

- To add a new field in the document after an existing field, hover over the row number in the dialog and click on the plus sign. The row number is not part of the document but is part of the dialog display.
- You can also add a new field at the end of the document by pressing the tab key when your text cursor is in the value of the last document field.

MODIFY AN EXISTING FIELD

- To modify documents, click on existing field names or values and make changes. In this example, the borough was changed from Manhattan to Queens. Changed fields appear highlighted in yellow:
- When you edit a document in List or Table view, Compass performs a findOneAndUpdate operation and updates only those fields that you have changed.

```
1   _id: ObjectId('5eb3d668b31de5d588f42942')                ObjectId
2   address : Object                                         Object
3     building : "502 "                                      String
4     coord : Array (2)                                      Array
5     street : "Amsterdam Avenue"                            String
6     zipcode : "10024"                                      String
    borough : "Queens"                                       String
8   cuisine : "Chicken"                                      String
9   grades : Array (6)                                       Array
10  name : "Harriet'S Kitchen"                               String
11  restaurant_id : "40362098"                               String
```

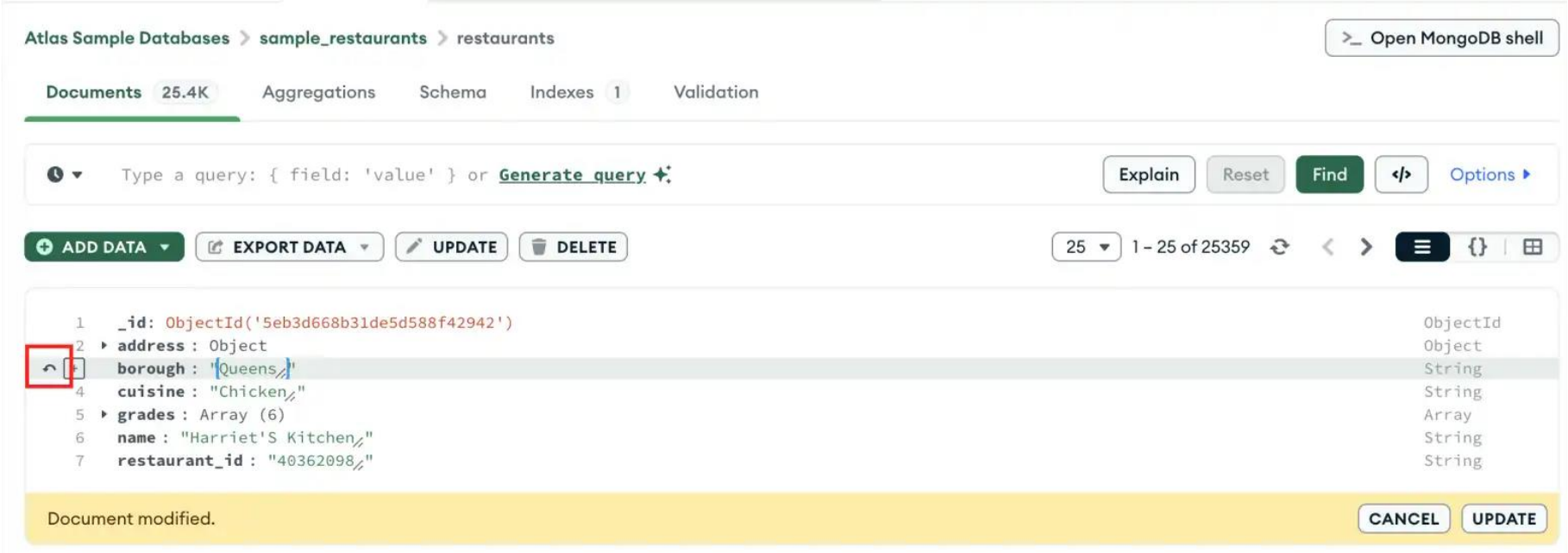
Document modified. CANCEL UPDATE

SAVE CHANGES

- When you are finished editing the document, click the Update button to commit your changes.

REVERT A CHANGE

- To revert changes to a document, hover over the edited field and click the revert icon which appears to the left of the field's line number.



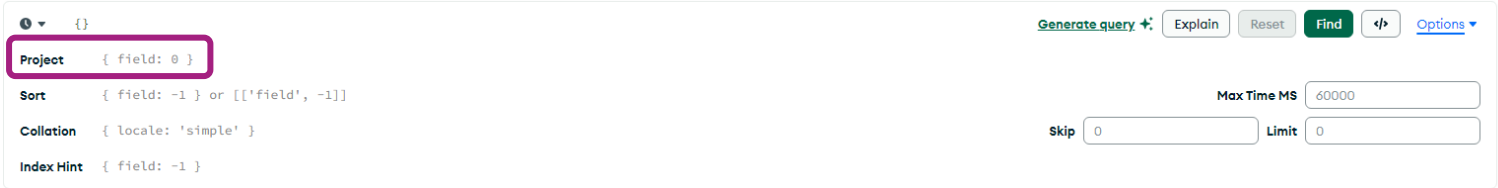
DELETE SINGLE DOCUMENT

- To delete a document, hover over the document and click the trash icon.
- After you click the delete button, the document is flagged for deletion. Compass asks for confirmation that you want to remove the document.
- Once you confirm, Compass deletes the document from the collection.

SET WHICH FIELDS ARE RETURNED

- If the query bar displays the Project option, you can specify which fields to return in the resulting data. By default, all fields are returned.
- To set a projection:

1. In the Query Bar, click Options.
2. Enter the projection document into the Project field.
   - ✓ To include fields: Specify the field name and set to 1 in the project document. **Example: { year: 1, name: 1 }** Only the fields specified in the project document are returned. The _id field is returned unless it is set to 0 in the Project document.
   - ✓ To exclude fields: Specify the field name and set to 0 in the project document. **Example: { year: 0, name: 0 }** All fields except for the fields specified in the project document are returned.
   - ✓ As you type, the Find button is disabled and the Project label turns red until a valid query is entered.
3. Click Find to run the query and view the updated results.



HOW DOES THE COMPASS QUERY COMPARE TO MONGODB AND SQL QUERIES?

EXAMPLE

You have 3,235 articles. You would like to see only the headlines and authors of those articles.

SQL

```
SELECT headline, author FROM article;
```

MongoDB Aggregation

```
db.article.aggregate(
  { $project : { headline : 1, author : 1 } }
);
```

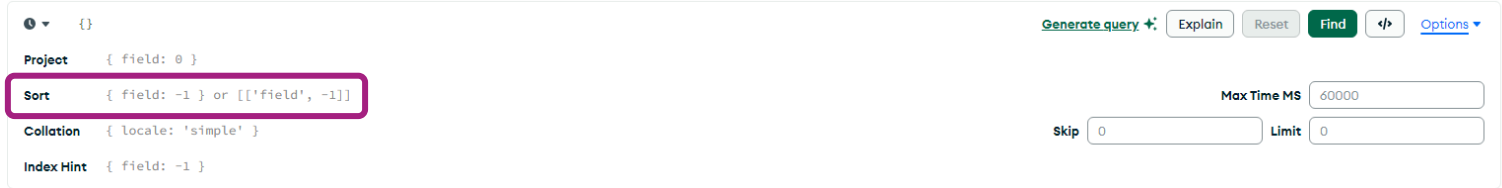
Compass Project Option

```
{ headline : 1, author : 1 }
```

SORT THE RETURNED DOCUMENTS

- If the query bar displays the Sort option, you can specify the sort order of the returned documents.
- To set the sort order:

1. In the Query Bar, click Options.
2. Enter the sort document into the Sort field.
   - To specify ascending order for a field, set the field to 1 in the sort document.
   - To specify descending order for a field, set the field and -1 in the sort documents.
   - Example: The following sort document sorts results first by year in descending order, and within each year, sort by name in ascending order. **{ year: -1, name: 1 }** As you type, the Find button is disabled and the Sort label turns red until a valid query is entered.
3. Click Find to run the query and view the updated results.



HOW DOES THE COMPASS QUERY COMPARE TO MONGODB AND SQL QUERIES?

EXAMPLE

You have 3,235 articles. You would like to see a list of articles sorted alphabetically by headline.

SQL

```
SELECT * FROM article
ORDER BY headline ASC;
```

MongoDB Aggregation

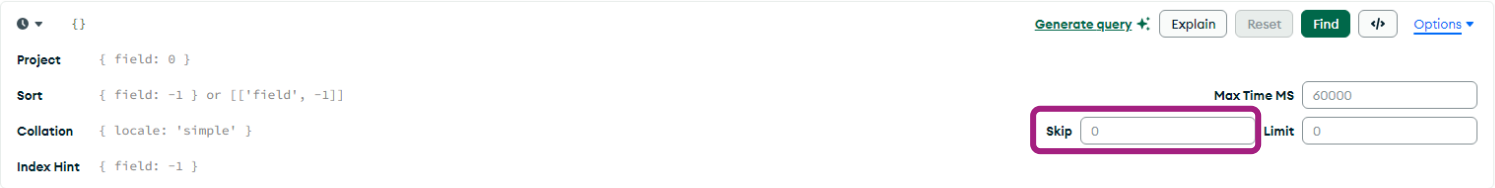
```
db.article.aggregate(
  { $sort : { headline : 1 } }
);
```

Compass Sort Option

```
$sort : { headline : 1 }
```

SKIP A NUMBER OF DOCUMENTS

- To specify the number of documents to skip:
  1. In the Query Bar, click Options.
  2. Enter an integer representing the number of documents to skip into the Skip field.
  3. Click Find to run the query and view the updated results.



HOW DOES THE COMPASS QUERY COMPARE TO MONGODB AND SQL QUERIES?

EXAMPLE

You have a 3,235 articles. You would like to see a list of articles grouped in blocks of 50, starting with the 436th record.

SQL

```
SELECT * FROM article
LIMIT 50 OFFSET 435;
```

MongoDB Aggregation

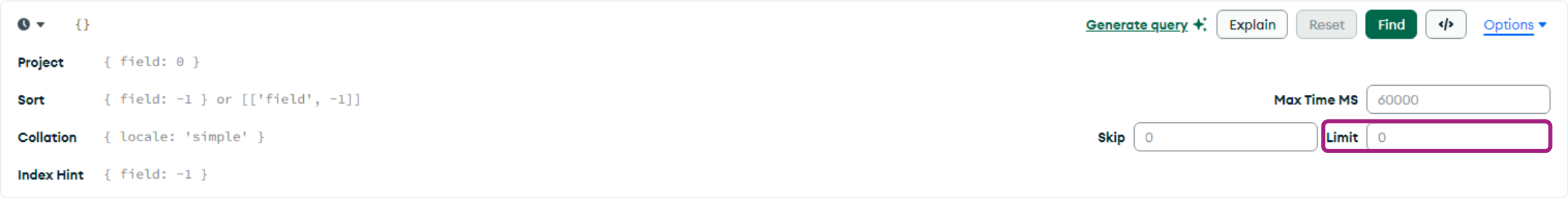
```
db.article.aggregate(
  { $limit : 50 },
  { $skip : 435 }
);
```

Compass Skip Option

```
$skip : 435
```

LIMIT THE NUMBER OF RETURNED DOCUMENTS

- To specify the limit:
  1. In the Query Bar, click Options.
  2. Enter an integer representing the number of documents to return into the Limit field.
  3. Click Find to run the query and view the updated results.



HOW DOES THE COMPASS QUERY COMPARE TO MONGODB AND SQL QUERIES?

EXAMPLE

You have 3,235 articles. You would like to see a list of the first 10 articles.

SQL

```
SELECT * FROM article
LIMIT 10;
```

MongoDB Aggregation

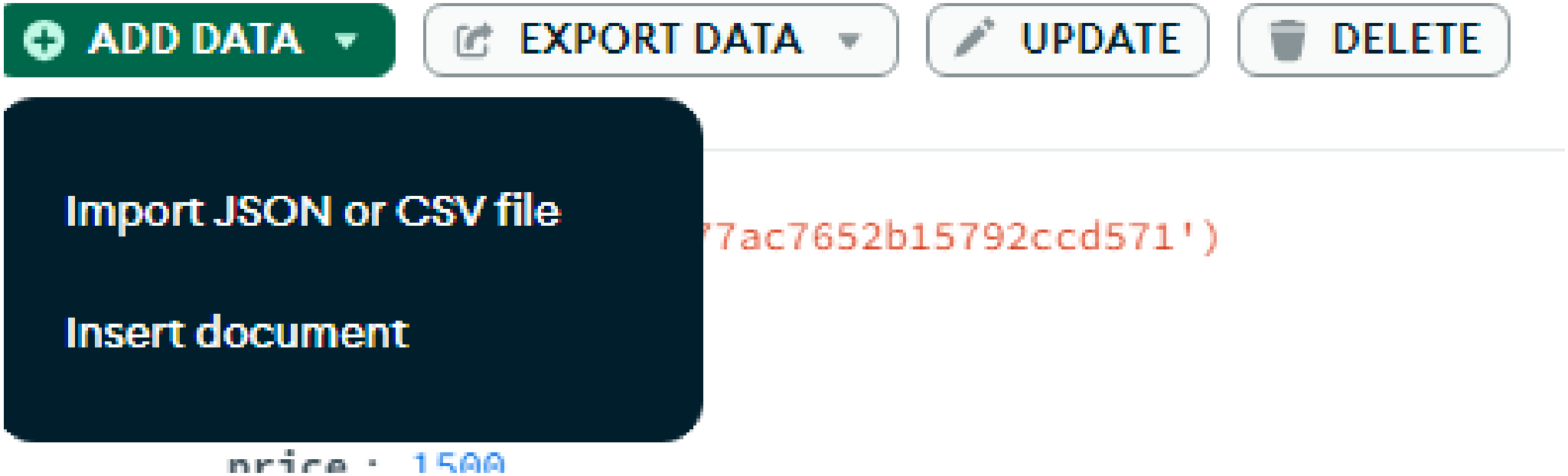
```
db.article.aggregate(
  { $limit : 10 }
);
```

Compass Limit Option

```
$limit : 10
```

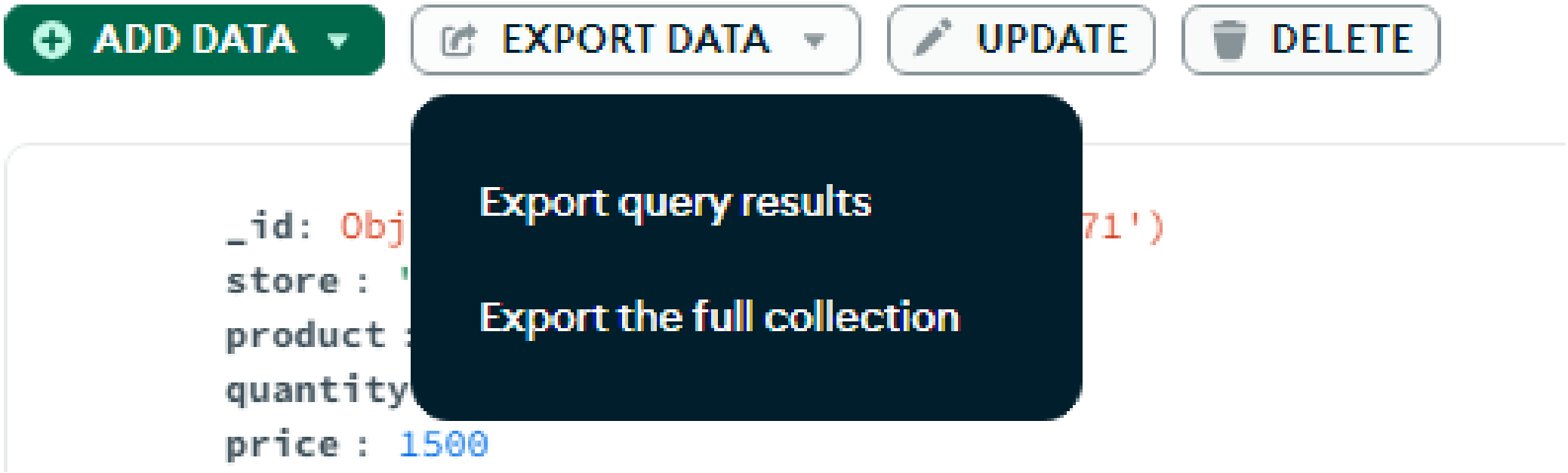
IMPORT DATA INTO A COLLECTION

- MongoDB Compass can import data into a collection from either a JSON or CSV file.



EXPORT DATA FROM A COLLECTION

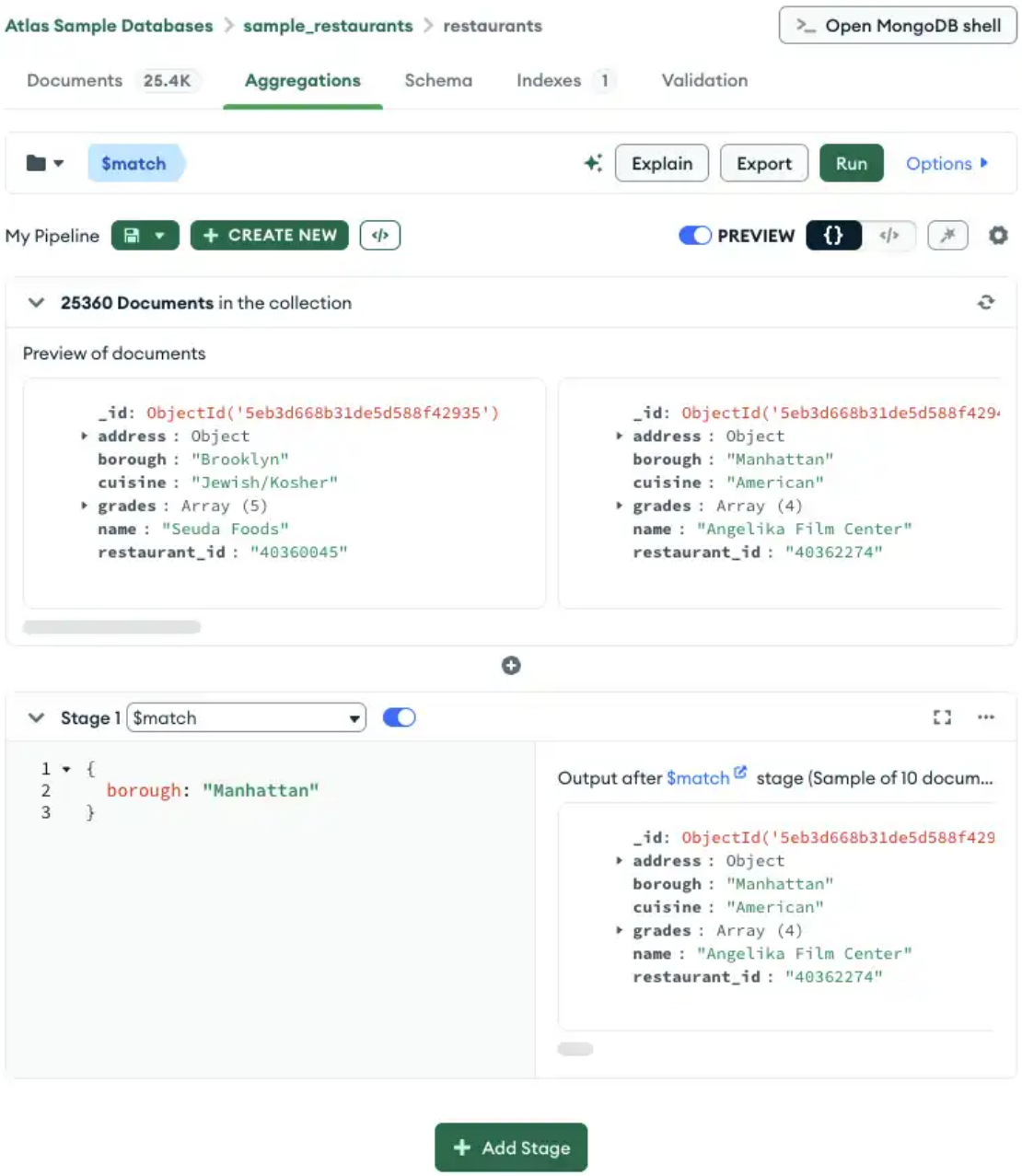
- MongoDB Compass can export data from a collection as either a JSON or CSV file. If you specify a filter or aggregation pipeline for your collection, Compass only exports documents which match the specified query or pipeline results



AGGREGATION PIPELINE IN MONGO COMPASS

CREATE AN AGGREGATION PIPELINE

- The Aggregation Pipeline Builder in MongoDB Compass helps you create aggregation pipelines to process documents from a collection or view and return computed results.
- To build an aggregation pipeline, choose a collection and click the **Aggregations** tab. Compass displays a blank aggregation pipeline. The **Preview of Documents in the Collection** section shows 10 documents randomly sampled from the chosen collection.



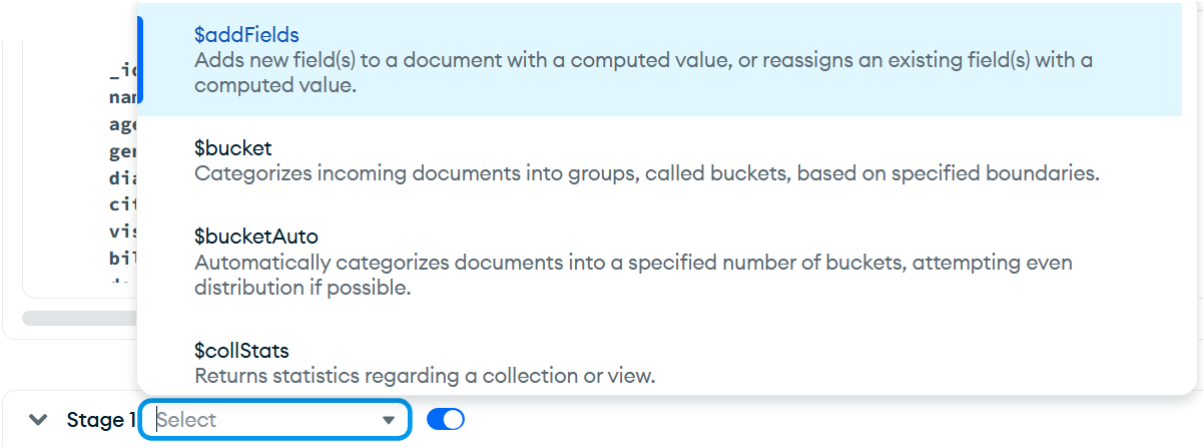
MONGODB COMPASS PROVIDES DIFFERENT MODES TO CREATE AGGREGATION PIPELINES:

- Stage View Mode, a visual pipeline editor that preloads pipeline syntax based on your selected stages.
- Stage Wizard, a feature of Stage View Mode that provides a set of templates for simple aggregation stage use cases. The Stage Wizard only includes simple use cases to help you get started with your aggregation pipeline.
- Focus Mode, a feature of Stage View Mode where you edit one pipeline stage at a time. Focus Mode helps you manage complex or deeply nested aggregation pipeline stages.
- Text View Mode, a text-based pipeline editor that accepts raw pipeline syntax.

STAGE VIEW MODE

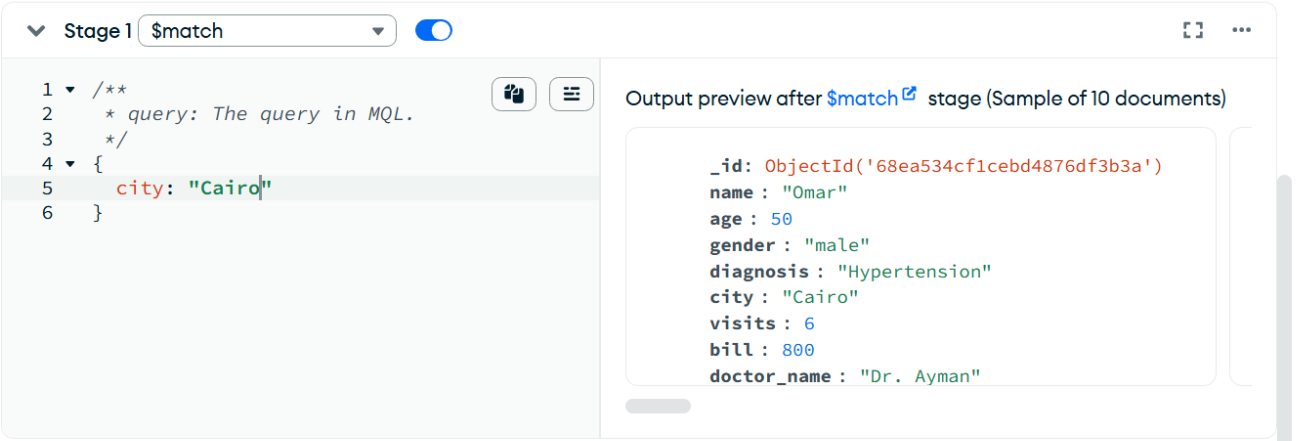
1. **Add an aggregation stage:** At the bottom of the aggregation pipeline pane, click the **+ Add Stage** button.

2. **Select an aggregation pipeline stage:** On the upper-left corner of the aggregation stage card, click the Select drop-down menu and select the aggregation pipeline stage to use for the first stage of the pipeline.

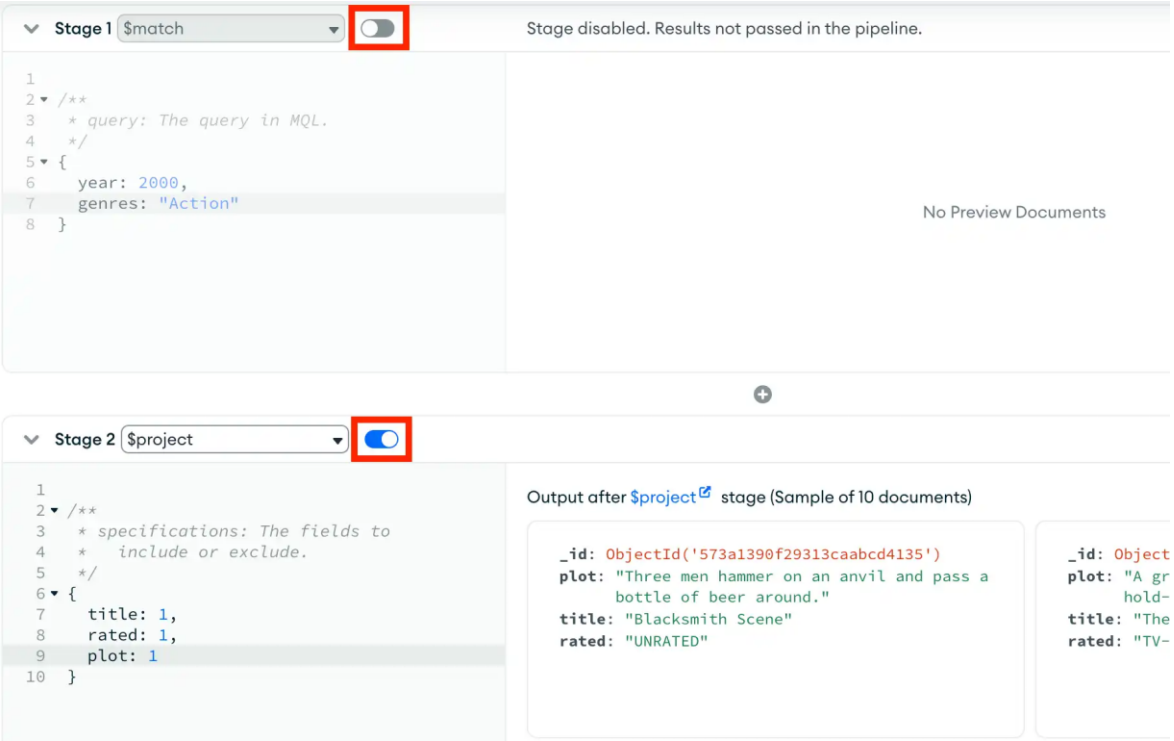


STAGE VIEW MODE

3. Fill in your pipeline stage: Fill in your selected stage. You can adjust the width of the pipeline stage by dragging its border to the right.



- The toggle to the right of each pipeline stage name dictates whether that stage is included in the pipeline. Toggling a pipeline stage also updates the pipeline preview, which updates based on whether or not that stage is included.
- For example, the following pipeline excludes the first $match stage and only includes the $project stage:



STAGE VIEW MODE

4. Add additional pipeline stages:
   - To add an additional pipeline stage after your last aggregation stage, click Add Stage. To add an aggregation stage before your most recently added stage, click the + icon above the stage card.
   - Repeat steps 2 and 3 for each additional stage.
   - You can change the order of pipeline stages by dragging the header of each stage card.
5. Run the pipeline: At the top-right corner of the pipeline builder, click **Run**. Compass returns your results in the document view.

STAGE WIZARD

1. **(Optional) Search for an aggregation use case:** On the Stage Wizard card, you can filter the use cases by searching for keywords associated with the use case or aggregation stage.
2. **Select an aggregation pipeline stage use case:** On the **Stage Wizard** card, select a stage use case for the first stage of your pipeline. You can click the stage card to add it to the end of your pipeline or drag it to your preferred position. After you select a use case, Compass populates the stage card with a form that corresponds to the selected aggregation pipeline stage. The Stage Wizard use cases include the following aggregation stages: $group, $lookup, $match , $project , $sort
3. **Fill in your pipeline stage:** Fill in the form for your selected stage and click **Apply**. After you click **Apply**, the form will turn into a stage card that you can edit in Stage View Mode, Focus Mode, or Text View Mode. Compass populates the Stage Output with up to ten sample output documents.
   - You cannot edit an existing stage through the Stage Wizard. The Stage Wizard can only add new stages. To edit an existing stage, use Stage View Mode, Focus Mode, or Text View Mode.
4. Add additional pipeline stages: To add more aggregation stages to your pipeline, repeat steps 3 and 4 for each additional stage.
5. Run the pipeline: At the top-right corner of the pipeline builder, click Run. Compass returns your results in the document view.

Find all the documents that match one or more conditions $match

AND OR

+ NESTED GROUP

city = cairo String

Cancel Apply

Stage Wizard

Search for a Stage

Find all the documents that match one or more conditions $MATCH

Group my documents based on their field values $GROUP

Compute values within the groups I create $GROUP

FOCUS MODE

sample_restaurants.restaurants

25.4k DOCUMENTS 1 INDEXES

Documents | Aggregations | Schema | Explain Plan | Indexes | Validation

Pipeline Your pipeline is currently empty. Explain Export Run More Options

Untitled – *modified* SAVE CREATE NEW EXPORT TO LANGUAGE PREVIEW STAGES TEXT

```
grades: Array
name: "Riviera Caterer"
restaurant_id: "40356018"
```

```
grades: Array
name: "Wilken'S Fine Food"
restaurant_id: "40356483"
```

```
grades: Array
name: "Kosher Island"
restaurant_id: "40356442"
```

Stage 1 Select

1

No Preview Documents

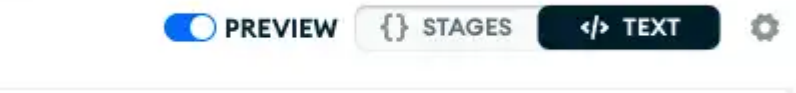
FOCUS MODE

1. **Select an aggregation pipeline stage:** Click the **Select** drop-down menu and select the aggregation pipeline stage to use for the first stage of the pipeline.

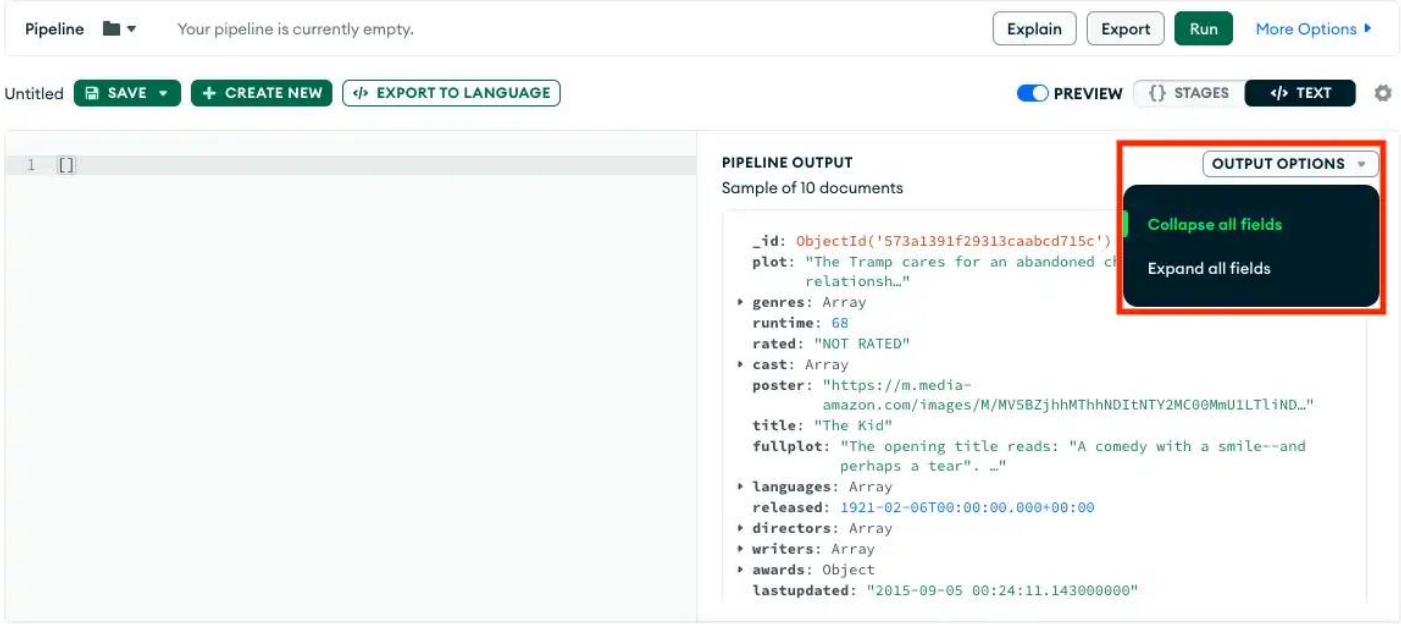
2. **Fill in your pipeline stage:** Fill in your selected stage. Compass populates the **Stage Output** with up to ten sample output documents. You can adjust the width of the **Stage Input**, stage editor, and the **Stage Output** by dragging their border to the desired size.
3. **Add additional pipeline stages:** Click the **Add Stage** dropdown to add additional aggregation stages before or after your last aggregation stage. Repeat steps 4 and 5 for each additional stage.
4. **Navigate between stages:** To navigate between different stages, select the stage you want to edit from the **Stage** dropdown in the upper-left corner of the Focus Mode modal.

5. **Run the pipeline:** Click **x** to exit Focus Mode and select **Run** at the top right of the pipeline builder. Compass returns your results in the document view.

TEXT VIEW MODE



1. **Enter your aggregation pipeline:** Enter valid aggregation syntax into the text editor. The text editor provides real-time linting for correct syntax and debugging information. You can also use Text View Mode to import aggregation pipelines from plain text by typing or pasting your pipeline into the text editor.
2. **Run the pipeline:** Click **Run** at the top right of the pipeline builder. Compass returns your results in the document view.

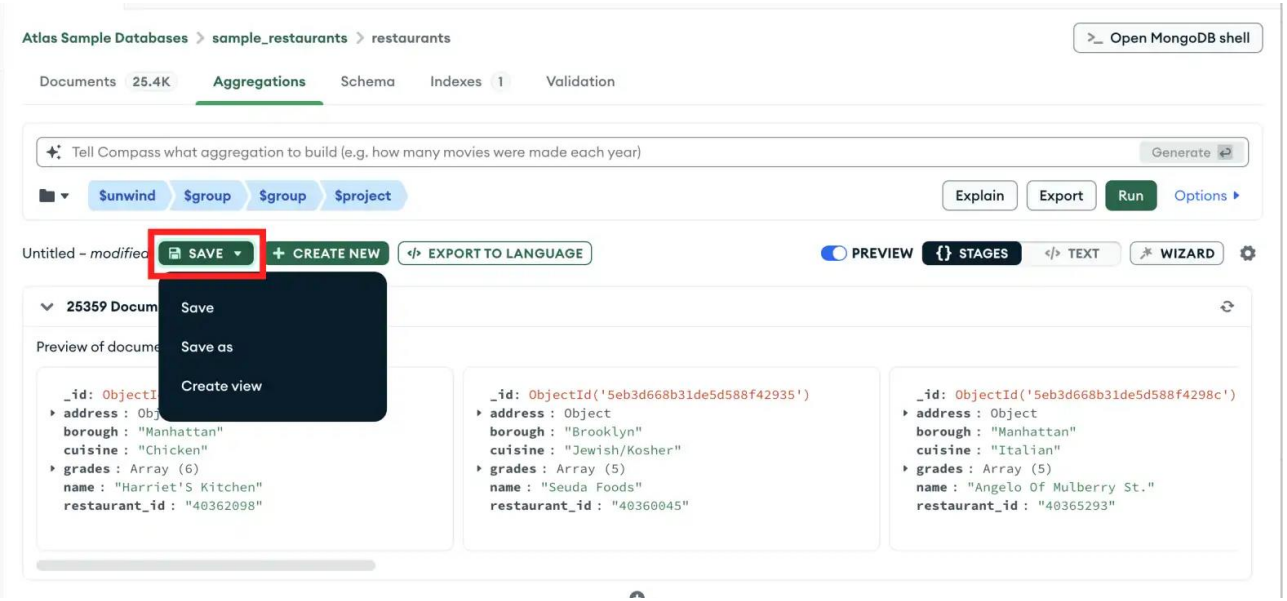


To expand all embedded fields and documents within the preview results, click Output Options and select Expand all fields.

SAVE A PIPELINE

- You can save your aggregation pipelines to find and use them again later. If you load a saved pipeline, you can make edits to the pipeline stages without changing the original saved pipeline. You can also create a view from your pipeline results.

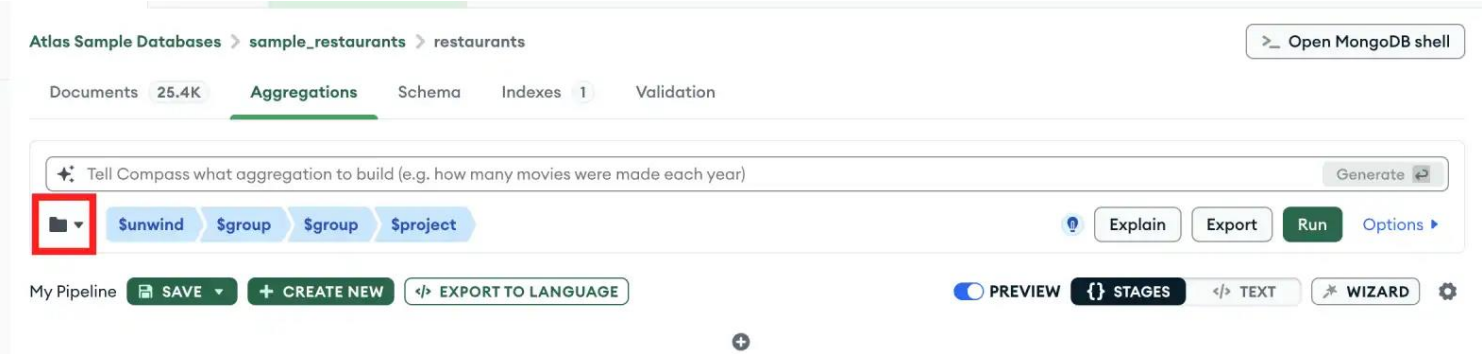
1. Click the save dropdown button: In the Aggregations pane, click the Save drop-down menu and select Save as.
2. Enter a name for your pipeline
3. Save the pipeline: Click the Save button to save your pipeline. Your pipeline will be saved under the folder icon at the top left of the pipeline builder.



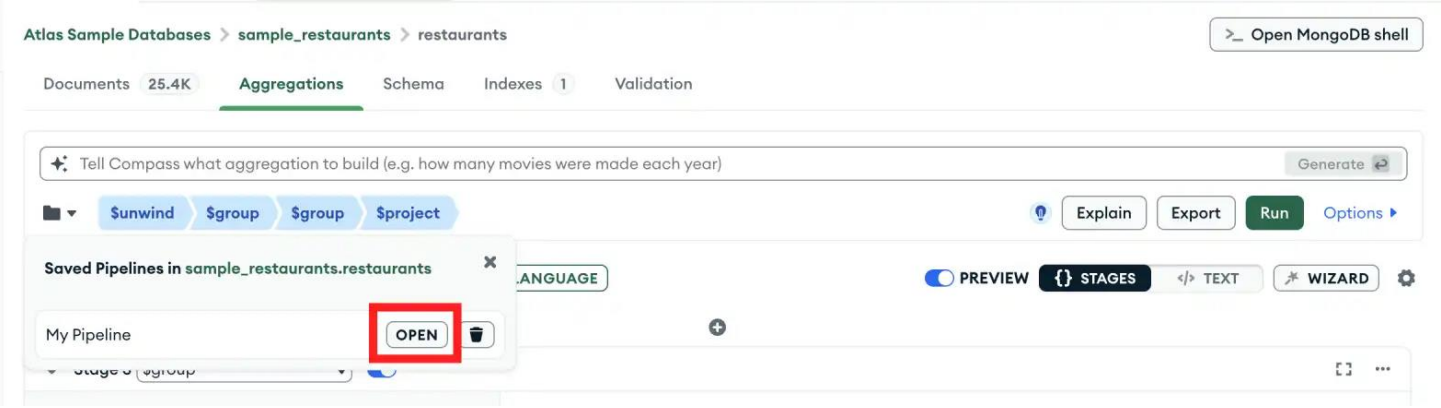
OPEN A SAVED PIPELINE

- Instead of creating a new pipeline from the beginning, you can load and edit saved aggregation pipelines.

1. **Click the folder icon:** In the top-left corner of the pipeline builder, click the folder icon to open the drop-down menu of saved pipelines.



2. Select an aggregation pipeline: In the drop-down menu, hover over the pipeline you want to open and click Open.



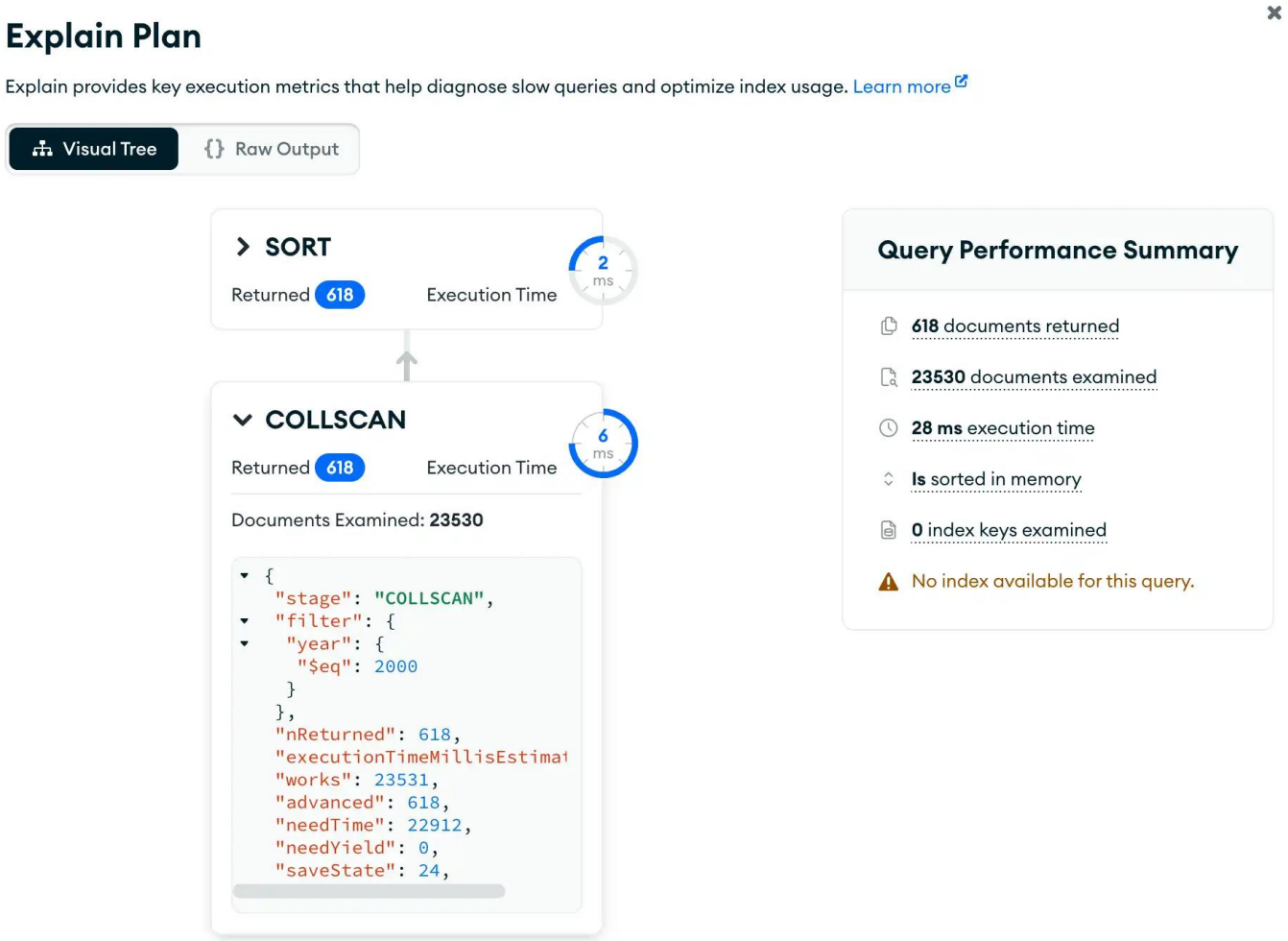
3. Confirm your selection: In the pop-up modal, click Open Pipeline.

VIEW EXPLAIN PLANS FOR A PIPELINE

- To help you better understand the performance of your pipeline, you can view your pipeline's explain plan. You can view the explain plan at any point while creating or editing your pipeline.
- On the Explain modal, you can view the explain stages as a Visual Tree, where each stage of the pipeline appears as a node on the tree. Alternatively, you can view the explain details in raw JSON format by selecting the Raw Output view.

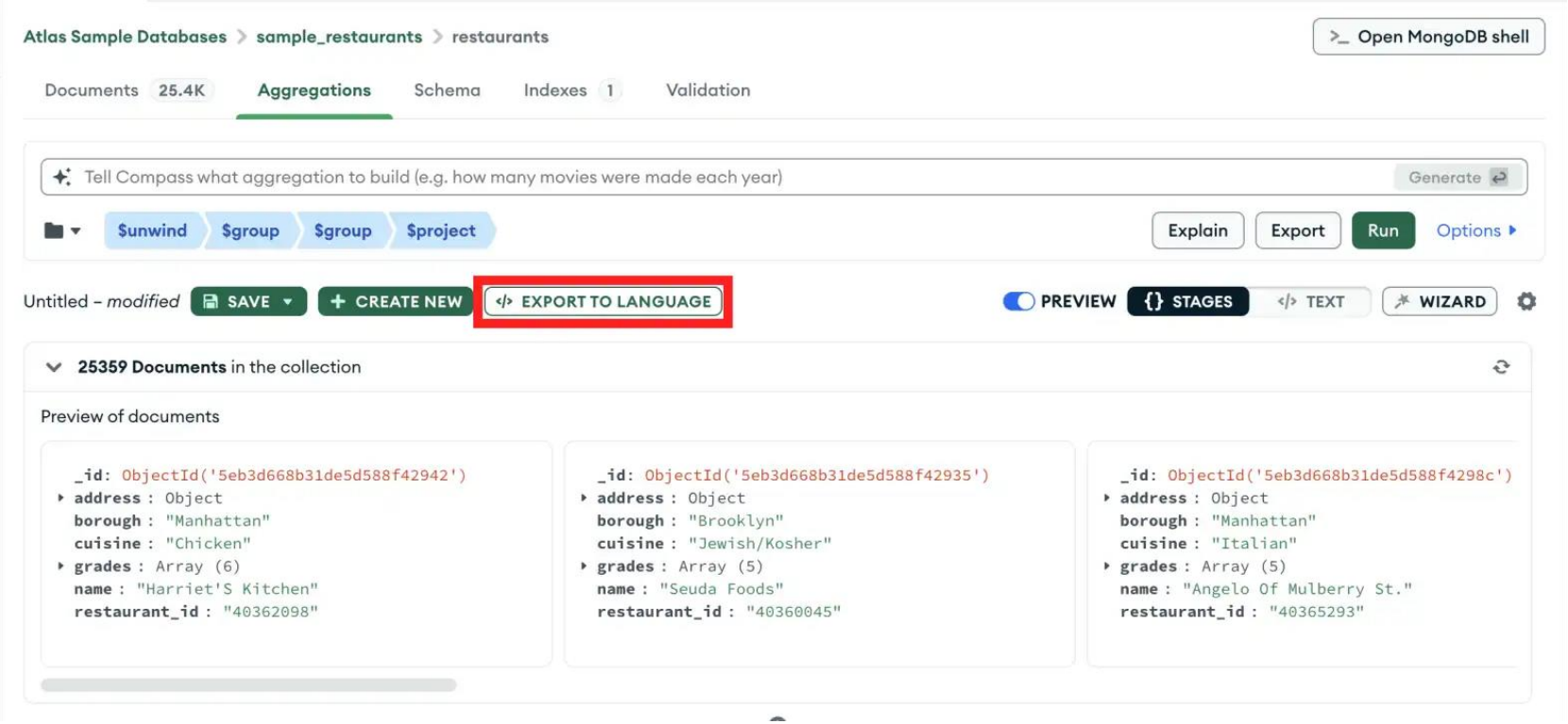
VIEW EXPLAIN PLANS FOR A PIPELINE

- The explain plan includes a **Query Performance Summary** with information on the execution of your aggregation pipeline such as: Execution time, The number of returned documents, The number of examined documents, The number of examined index keys
- The explain stages are are shown as a Visual Tree. Each stage of the pipeline appears as a node on the tree. You can click on each stage for more detailed execution information about the stage.
- (Optional) Select the Raw Output view: To view your full explain plan as raw JSON, select the Raw Output view.



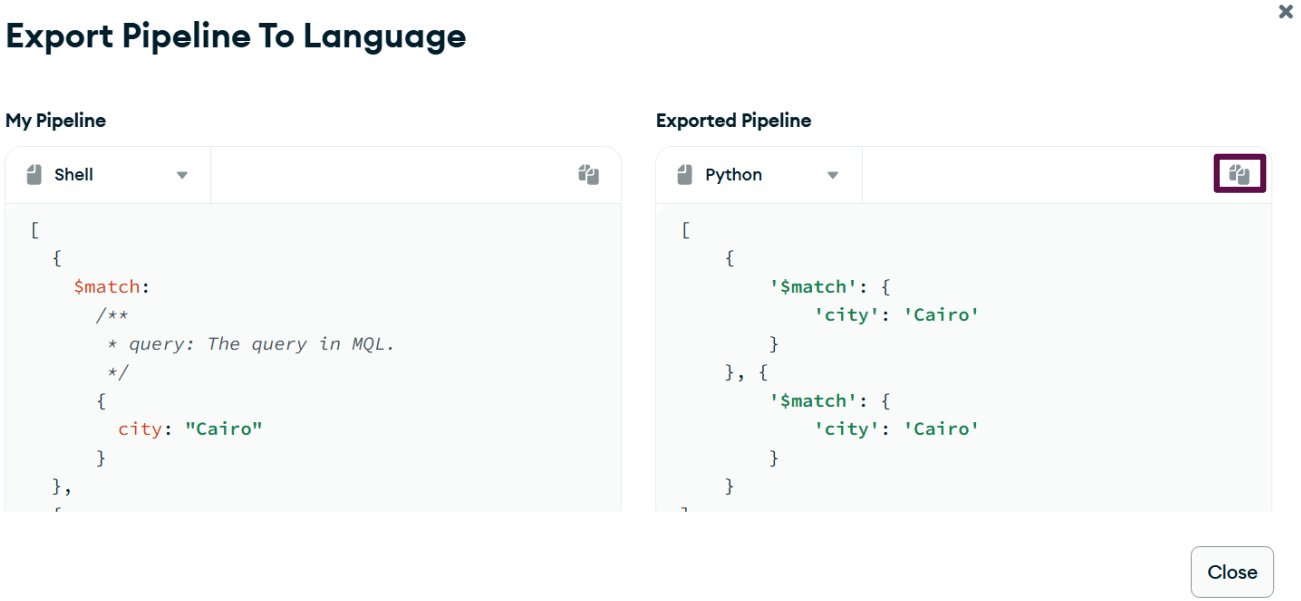
EXPORT PIPELINE TO SPECIFIC LANGUAGE

- You can use the Aggregation Pipeline Builder to format and export finished pipelines. You can export pipelines to a chosen language to use in your application.
- You can export your pipeline to the following languages: C#, Go, Java, Node, PHP, Python, Ruby
- In the aggregation pipeline pane, click the **Export to Language** button to open the pipeline export card.



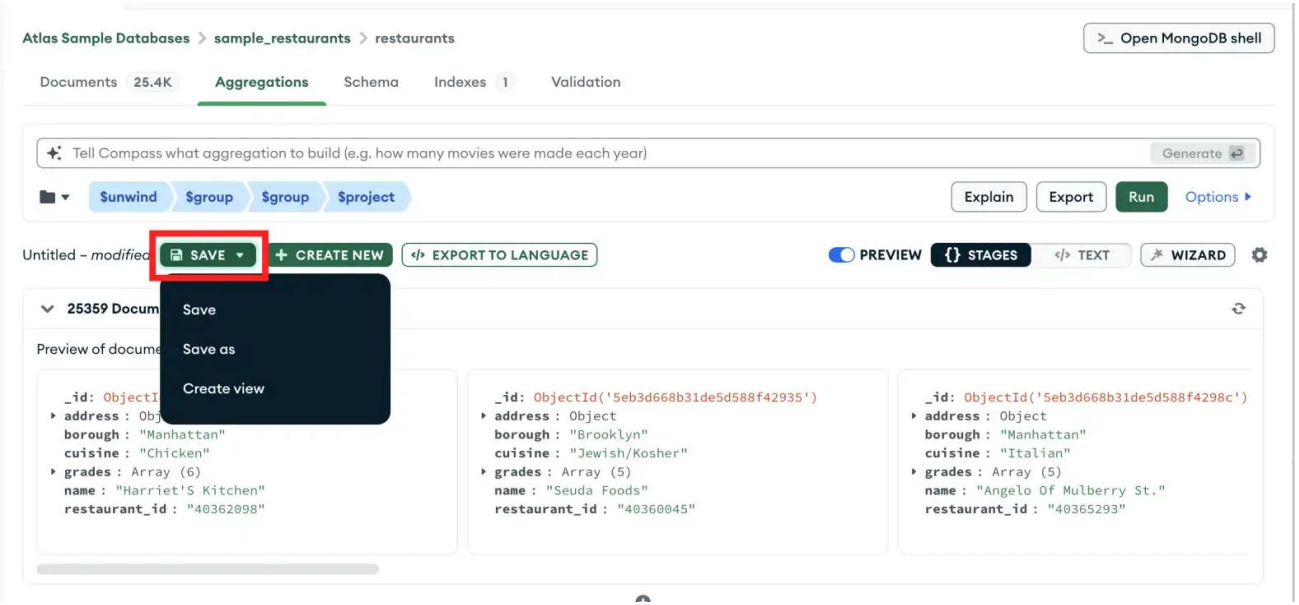
EXPORT PIPELINE TO SPECIFIC LANGUAGE

- Select your export language: On the right side of the card, click the drop-down menu under Exported Pipeline and select your desired programming language. The My Pipeline pane on the left of the export card displays your pipeline in mongosh syntax. The Exported Pipeline pane to the right displays your pipeline in the selected programming language.
- (Optional) Include import statements: Click the Include Import Statements checkbox to include the required import statements for the selected programming language.
- (Optional) Include driver syntax: Click the Include Driver Syntax checkbox to include application code for the selected programming language. If you include driver syntax, the copyable code reflects project, sort, maxtimems, collation, skip and limit options.
- Click the icon at the top-right of the pipeline to copy your pipeline for the selected programming language. You can now integrate and execute your created pipeline in your application.



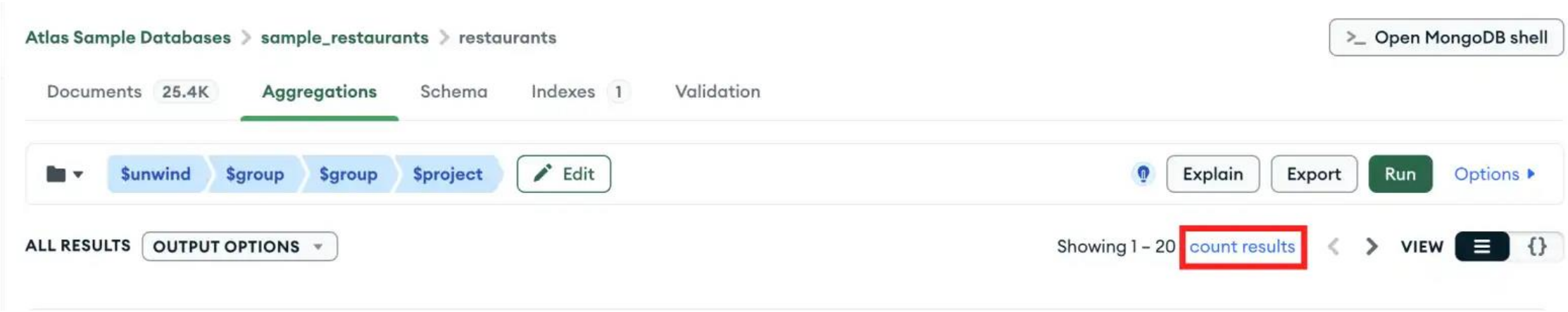
CREATE A VIEW FROM PIPELINE RESULTS

- To quickly access the results of an aggregation pipeline without having to run it, you can create a view on MongoDB Compass. Views are read-only, so they can help keep your data secure by limiting user access to a predefined set of results.
- In the aggregation pipeline pane, click the **Save** drop-down button and select **Create view**.
- **Enter a name for your view:** The view name must be between 6 and 1024 characters long.
- Click the **Create** button to create your view. Compass creates a view from your pipeline results in the same database where the pipeline was created.



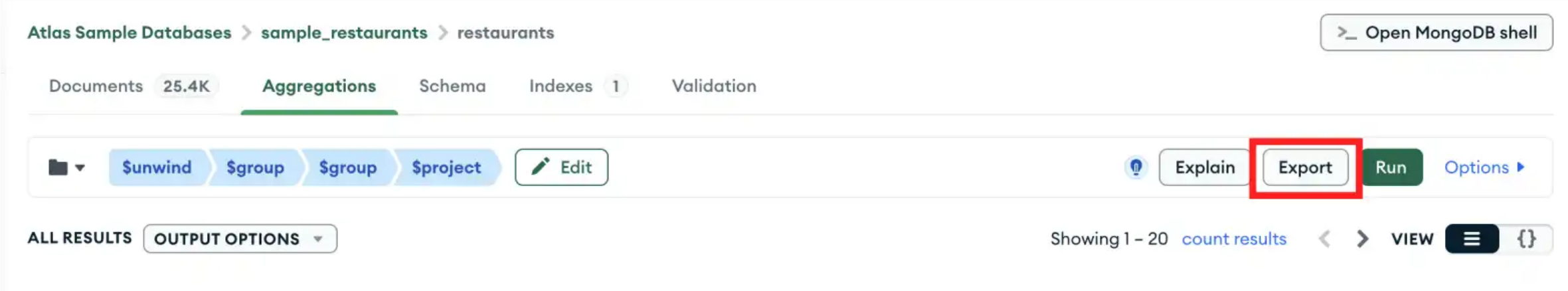
COUNT PIPELINE RESULTS DOCUMENTS

- You can view the number of documents outputted by your pipeline with the **count results** button.
- When you delete or add a document, you must manually refresh the **count results** value on the **Aggregations** tab to reflect the new document count.
- After you run your pipeline, click **count results**, which appears under the **Run** button. The **count results** button will update with the count of resulting documents.



EXPORT AGGREGATION PIPELINE RESULTS

- To use your aggregated data in other external tools, you can export your pipeline results from MongoDB Compass as a JSON or CSV file.
- Avoid using CSV files exports to back up your data. CSV files may lose type information and are not suitable for backing up your data.
- In the top right of the aggregation pipeline builder, click the **Export** button to open the modal.



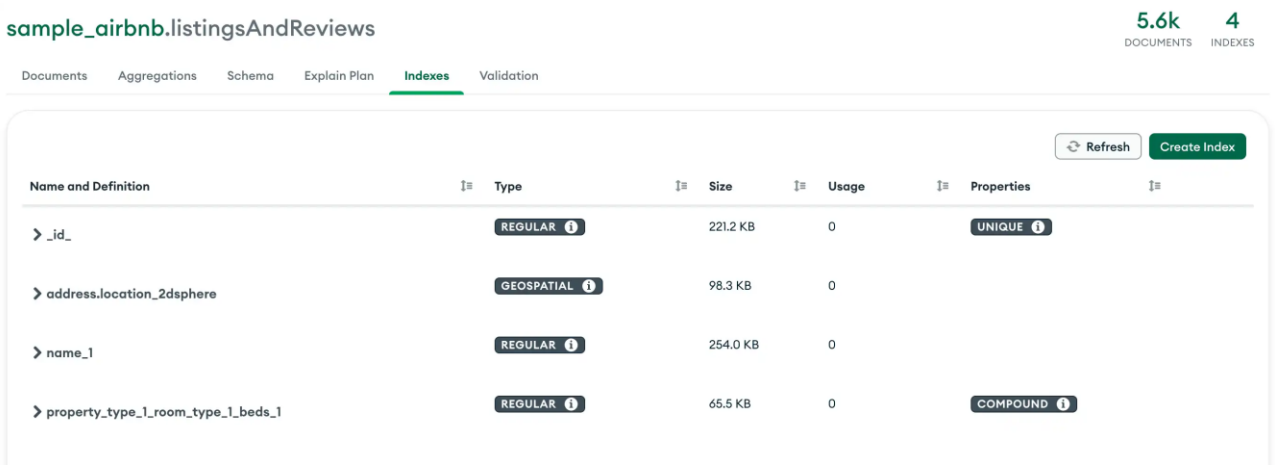
INDEXES IN MONGO COMPASS

MANAGE INDEXES

- Indexes are special data structures that improve query performance. Indexes store a portion of a collection's data in an easy-to-traverse form. The index stores the value of a specific field or set of fields, ordered by the value of the field.
- To improve query performance, build indexes on fields that appear often in queries and for all operations that sort by a field.
- Queries on an indexed field can use the index to limit the number of documents that must be scanned to find matching documents.
- Sort operations on an indexed field can return documents pre-sorted by the index

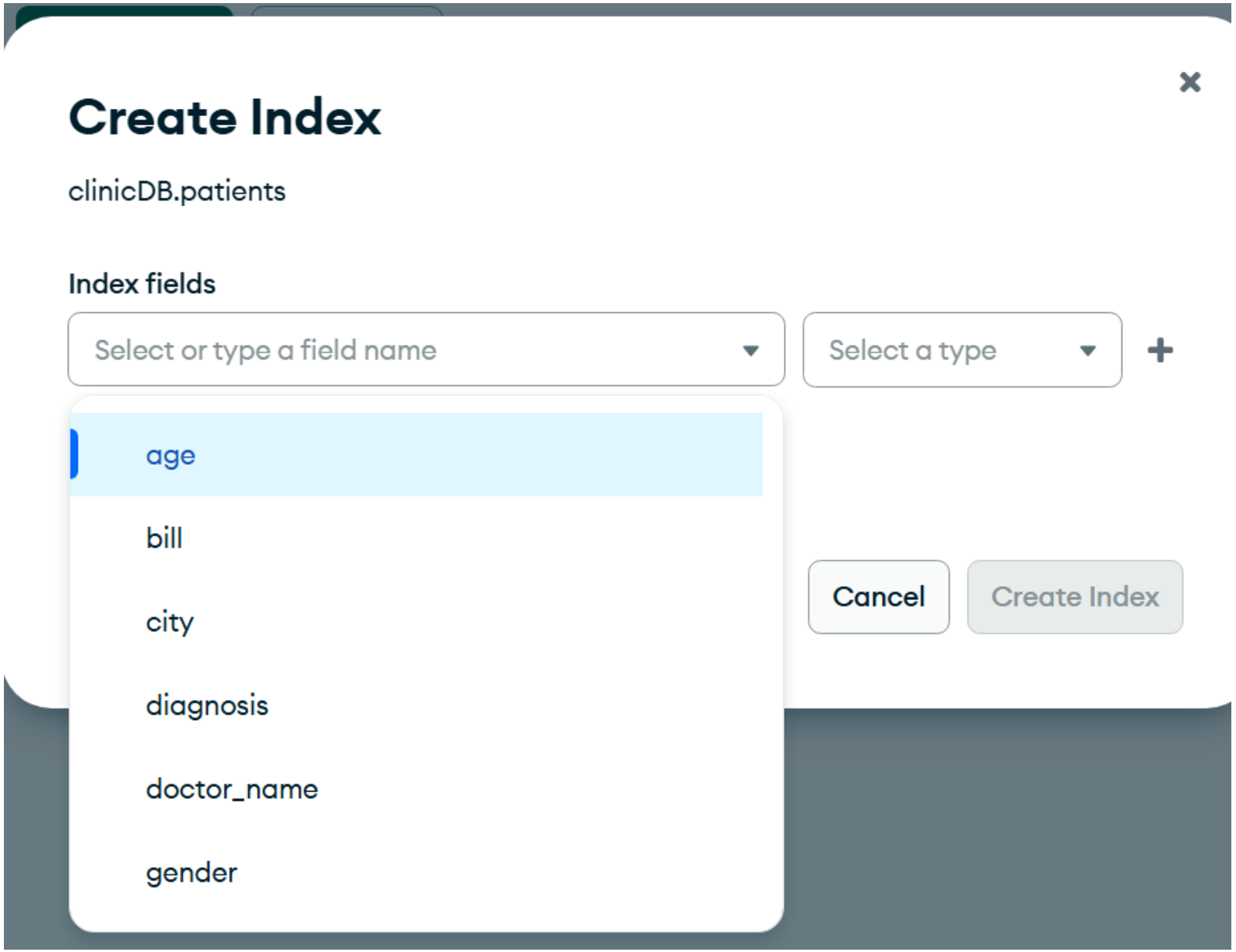
INDEXES TAB

- To access the **Indexes** tab for a collection, click on the collection on the left hand pane and select the **Indexes** tab.
- For each index, Compass displays the following information:
- Name and Definition: The name of the index and keys.
- Type: Regular, text, geospatial or hashed index.
- Size: How large the index is.
- Usage: Number of times the index has been used in a lookup since the time the index was created or the last server restart.
- Properties: Any special properties (such as uniqueness, partial) of the index.



CREATE AN INDEX

- From the Indexes tab, click the **Create Index** button.
- Add fields to the index
  - Specify an index key: To specify an existing document field as an index key, select the field from the dropdown list. To specify a field that does not exist in any document as an index key, enter the field name in the input box.
  - Use the dropdown to the right of each field name to specify the index type. You can specify one of the following types: Ascending, Descending



HIDE OR UNHIDE AN INDEX

- You can hide an index from the query planner to evaluate the potential impact of dropping an index without actually dropping the index.

Name & Definition	Type	Size	Usage	Properties	Status	
_id_	REGULAR	20.5 KB	1 (since Thu Oct 23 2025)	UNIQUE	READY	Hide Index
age_1	REGULAR	20.5 KB	0 (since Thu Oct 23 2025)		READY	

DROP AN INDEX

Name & Definition	Type	Size	Usage	Properties	Status
_id_	REGULAR	20.5 KB	1 (since Thu Oct 23 2025)	UNIQUE	READY
age_1	REGULAR	20.5 KB	0 (since Thu Oct 23 2025)		READY

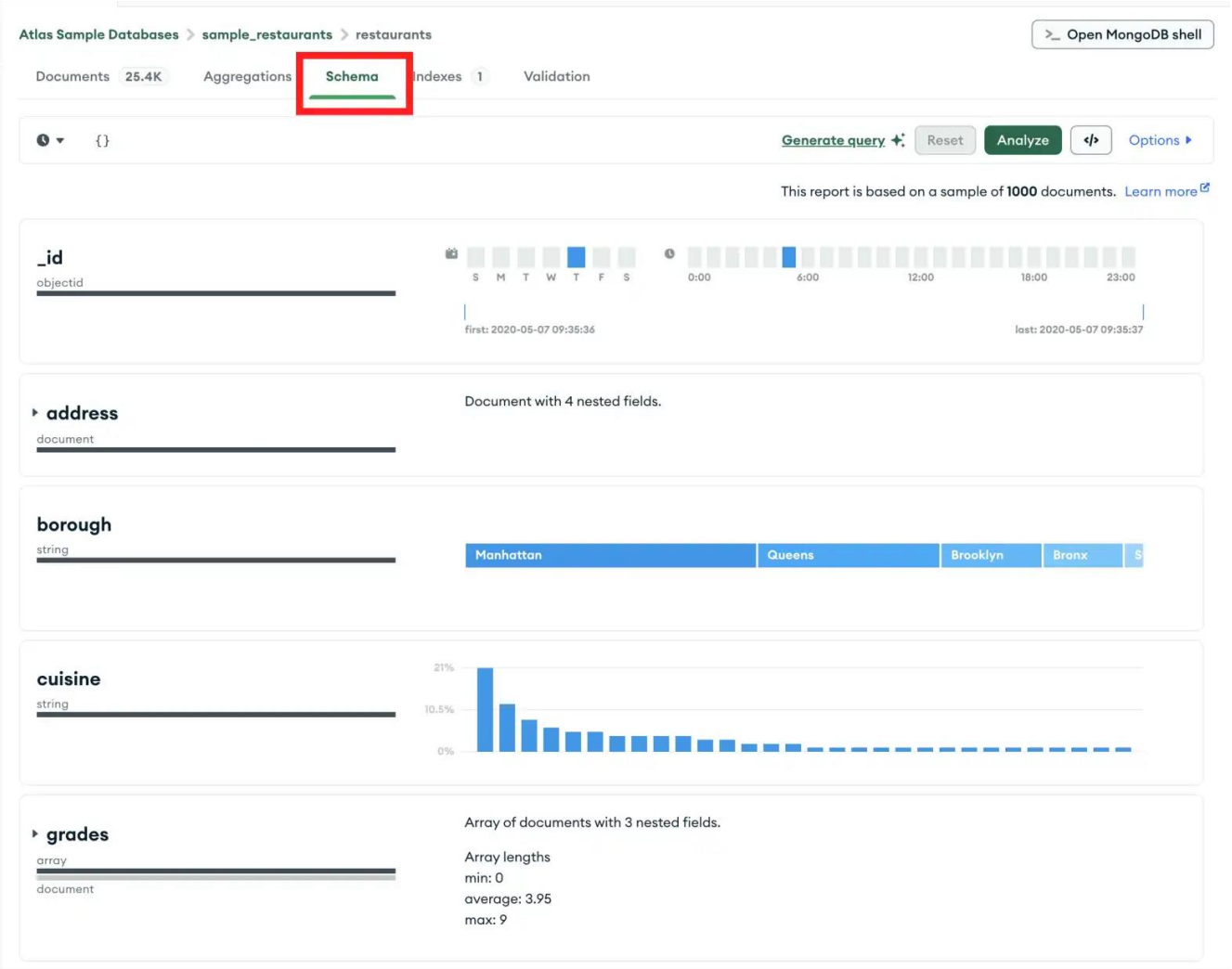
DATA SCHEMA IN MONGO COMPASS

ANALYZE YOUR DATA SCHEMA

- The Schema tab provides an overview of the data type and shape of the fields in a particular collection. Databases and collections are visible in the left-side navigation.
- The overview is based on sampling the documents in the collection. The schema overview may include additional data about the contents of the fields, such as the minimum and maximum values of dates and integers, the frequency of occurrence of particular values, and the cardinality of the data.
- MongoDB has a flexible schema model, which means that some fields may contain different types of data from one document to the next. For example, a field named address may contain strings and integers in some documents, objects in others, or some combination of all three.
- In the case of heterogenous fields, the Schema tab shows a breakdown of the various data types contained within the field with the percentage of each data type represented.

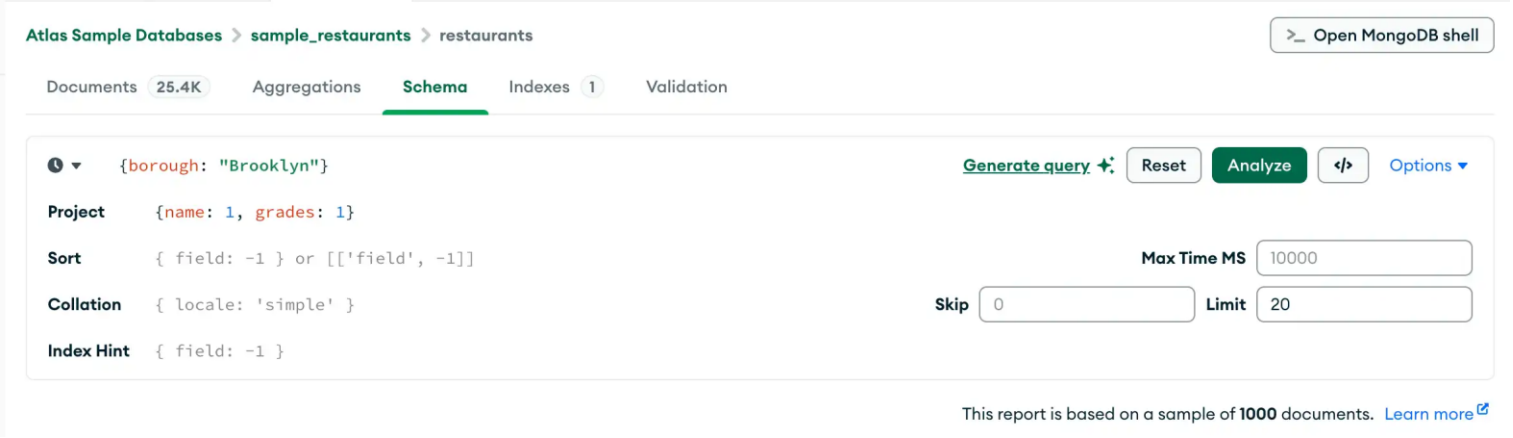
EXAMPLE

- The Schema tab shows size information about the test.restaurants collection at the top, including the total number of documents in the collection, the average document size, and the total disk space occupied by the collection.
- The following fields are shown with details:
- The _id field is an ObjectId. Each ObjectId contains a timestamp, so Compass displays the range of creation times for the sampled documents.
- The address field contains four nested fields. You can expand the field panel to see analyses of each of the nested fields.
- The borough field contains a string indicating the borough in which the restaurant is located. The cardinality is low enough that Compass can provide a graded bar of the field contents, with the most-frequently occurring string on the left.
- The grades field contains arrays of strings. The analysis shows the minimum, maximum, and average array lengths.



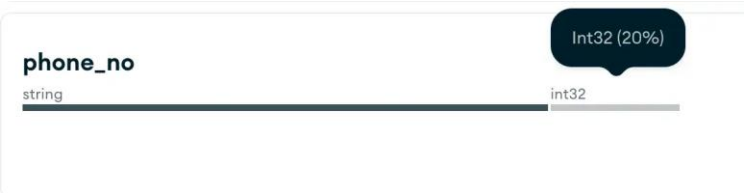
QUERY BAR

- Using the query bar in the Schema tab, you can create a query filter to limit your result set. Click the Options button to specify query options, such as the particular fields to display and the number of results to return.



FIELD DESCRIPTIONS

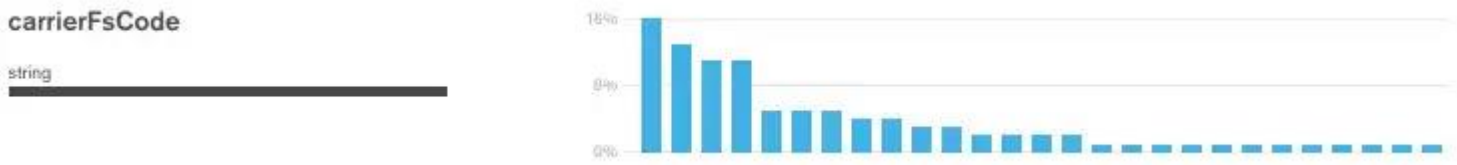
- For each field, Compass displays summary information about the data type or types the field contains and the range of values. Depending on the data type and the level of cardinality, Compass displays histograms, graded bars, geographical maps, and sample data to provide a sense of the shape and scope of the data contained in each field.
- **Field with a Single Data Type:** Below is an example of the data type summary for a field called last_login which contains data of type date.
- **Field with Multiple Data Types:** For fields that contain multiple data types, Compass displays a percentage breakdown of the various data types across documents. In the example below, the chart shows the contents of a field called phone_no in which 20% of documents are of type int32, and the remaining 80% are of type string.
- **Missing Field:** If a collection contains documents in which not all fields contain a value, the missing values display as undefined. In the example below, the field age has no recorded value in 40% of the sampled documents.



FIELD DESCRIPTIONS

- **Strings:** Strings can appear in three different ways. If there are entirely unique strings in a field, Compass shows a random selection of string values from the specified field. Click the circular refresh icon to see a new set of randomly selected values from the field.
- If there are only a few different string values, Compass shows the strings in a single graded bar which shows the percentage of the population of the string values.

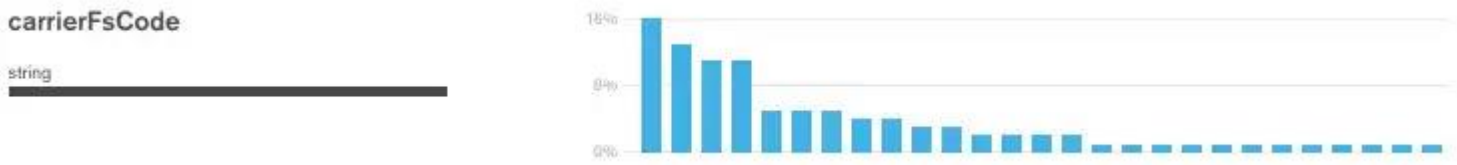
- If there are multiple string values with some duplicates, Compass shows a histogram indicating the frequency of each string found within the field.



FIELD DESCRIPTIONS

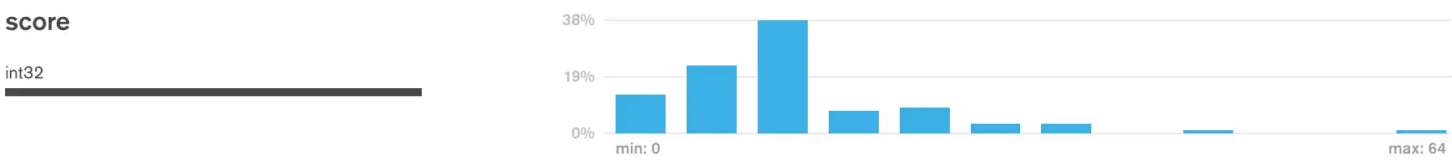
- **Strings:** Strings can appear in three different ways. If there are entirely unique strings in a field, Compass shows a random selection of string values from the specified field. Click the circular refresh icon to see a new set of randomly selected values from the field.
- If there are only a few different string values, Compass shows the strings in a single graded bar which shows the percentage of the population of the string values.

- If there are multiple string values with some duplicates, Compass shows a histogram indicating the frequency of each string found within the field.



FIELD DESCRIPTIONS

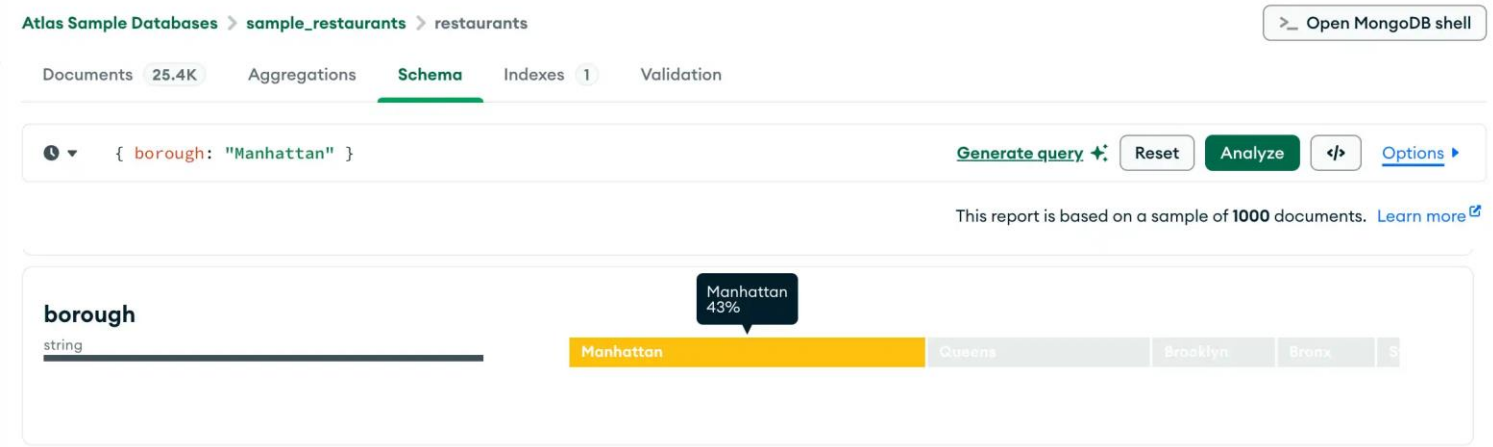
- **Numbers:** Numbers are similar to strings in their representation. Unique numbers are shown in the following manner. Duplicate numbers are shown in a histogram that indicates their frequency:



- **Dates and ObjectIDs:** Fields that represent dates (and fields that contain the ObjectID data type, which includes a timestamp) are shown across multiple bar charts. The two charts on the top row represent the day of the week and time of day of the timestamp value.

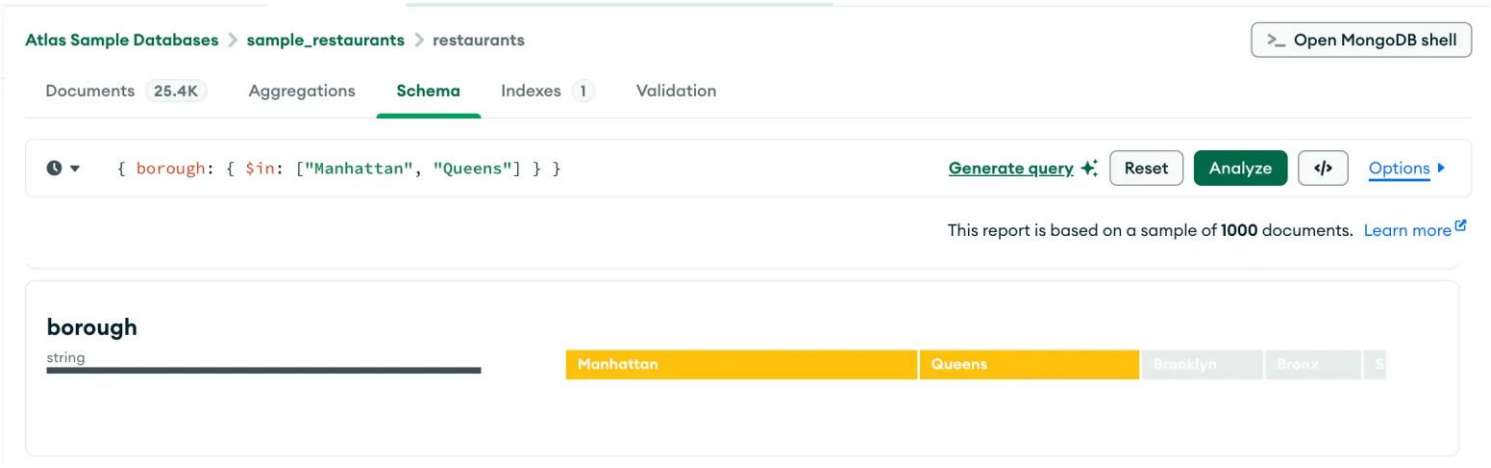
QUERY BUILDER

- In the **Schema** tab, you can type the filter manually into the query bar or generate the filter with the Compass query builder. The query builder allows you to select data elements from one or more fields in your schema and construct a query matching the selected elements.
- In the **Schema** view, you can click on a chart value to build a query. For example, the following image shows the query filter built by clicking the Manhattan value for the borough field.



QUERY BUILDER

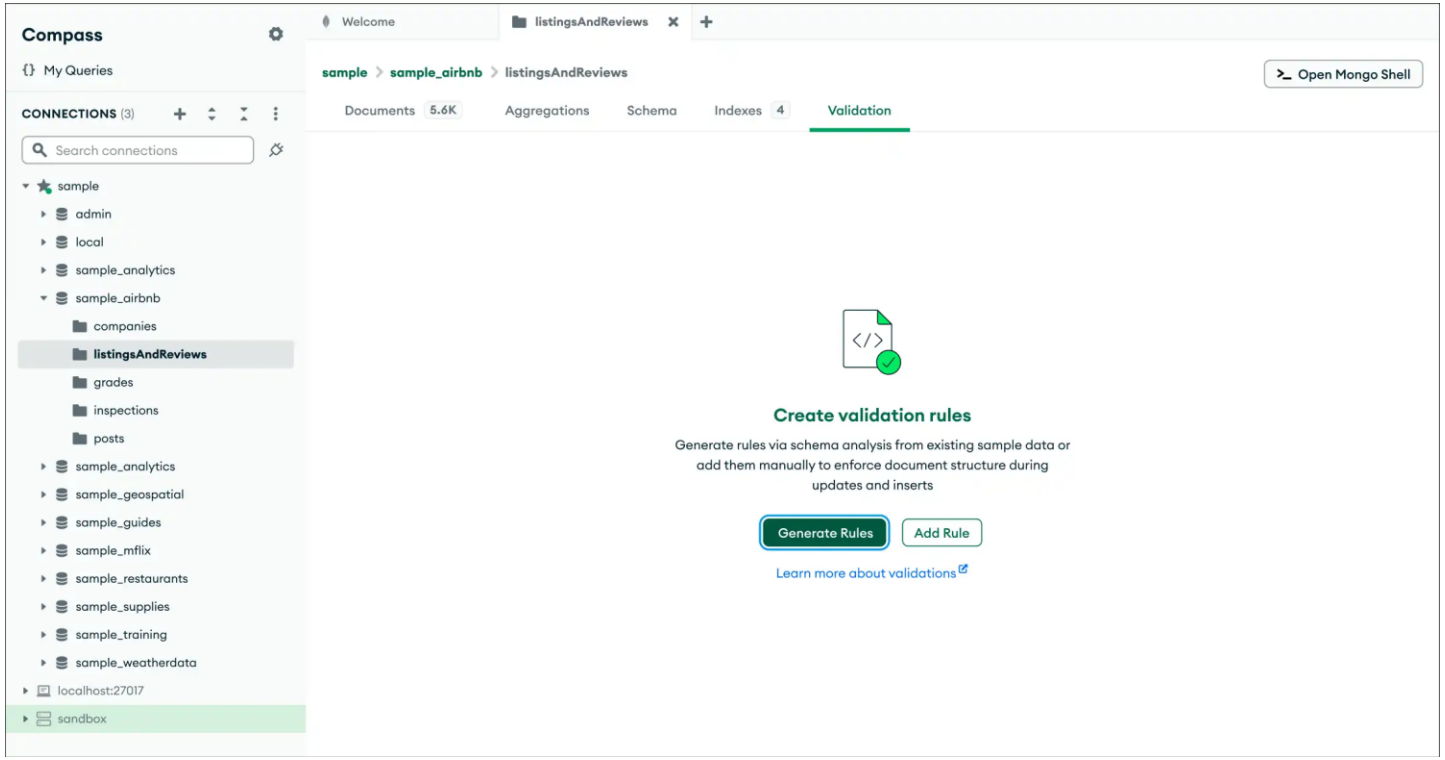
- To select multiple values for a field, click and drag the cursor over a selection of values, or press shift+click on the desired values.



VALIDATION RULES IN MONGO COMPASS

SET VALIDATION RULES FOR YOUR SCHEMA

- Schema validation ensures that all documents in a collection follow a defined set of rules, such as conforming to a specific shape or only allowing a specified range of values in fields.



JSON SCHEMA VALIDATION

- The $jsonSchema operator supports various keywords to specify validation rules. For example:
- The required array defines required fields in your document.
- The properties object defines rules for specific document fields.
- The list of required fields.
- The bsonType for all required fields.
- The minimum and maximum values in the address.coord array.
- The acceptable values for the borough field, using enum

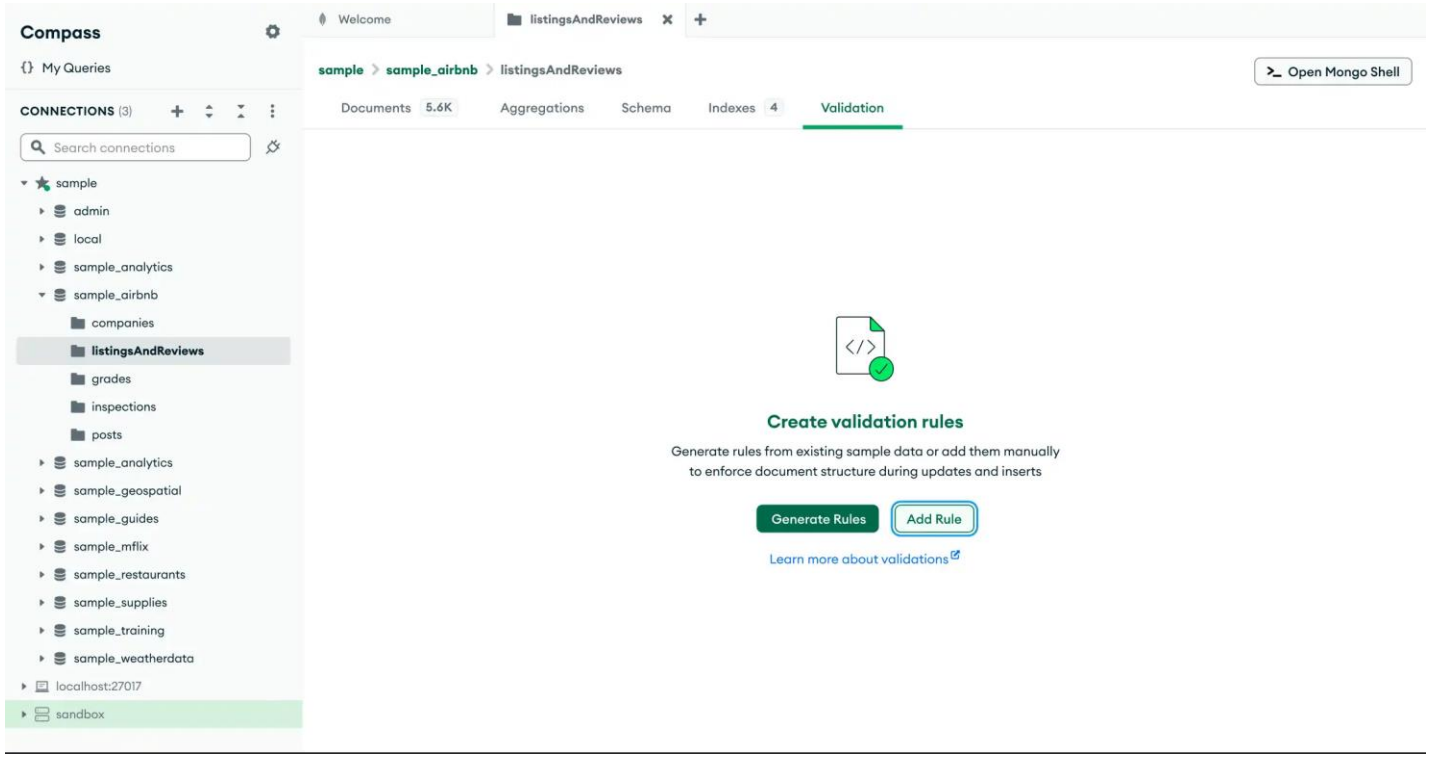
```
{
   $jsonSchema: {
      bsonType: "object",
      required: [ "address", "borough", "name" ],
      properties: {
         address: {
            bsonType: "object",
            properties: {
               coord: {
                  bsonType: "array",
                  items: [
                     {
                        bsonType: "double",
                        minimum: -180,
                        maximum: 180,
                        exclusiveMaximum: false,
                        description: "must be a number in [ -180, 180 ]"
                     },
                     {
                        bsonType: "double",
                        minimum: -90,
                        maximum: 90,
                        exclusiveMaximum: false,
                        description: "must be a number in [ -90, 90 ]"
                     }
                  ]
               }
            },
            description: "must be an object"
         },
         borough: {
            bsonType: "string",
            enum: [ "Manhattan", "Brooklyn", "Queens", "Bronx", "Staten Island"
            description: "must be one of the enum strings"
         }
```

VALIDATION USING QUERY OPERATORS

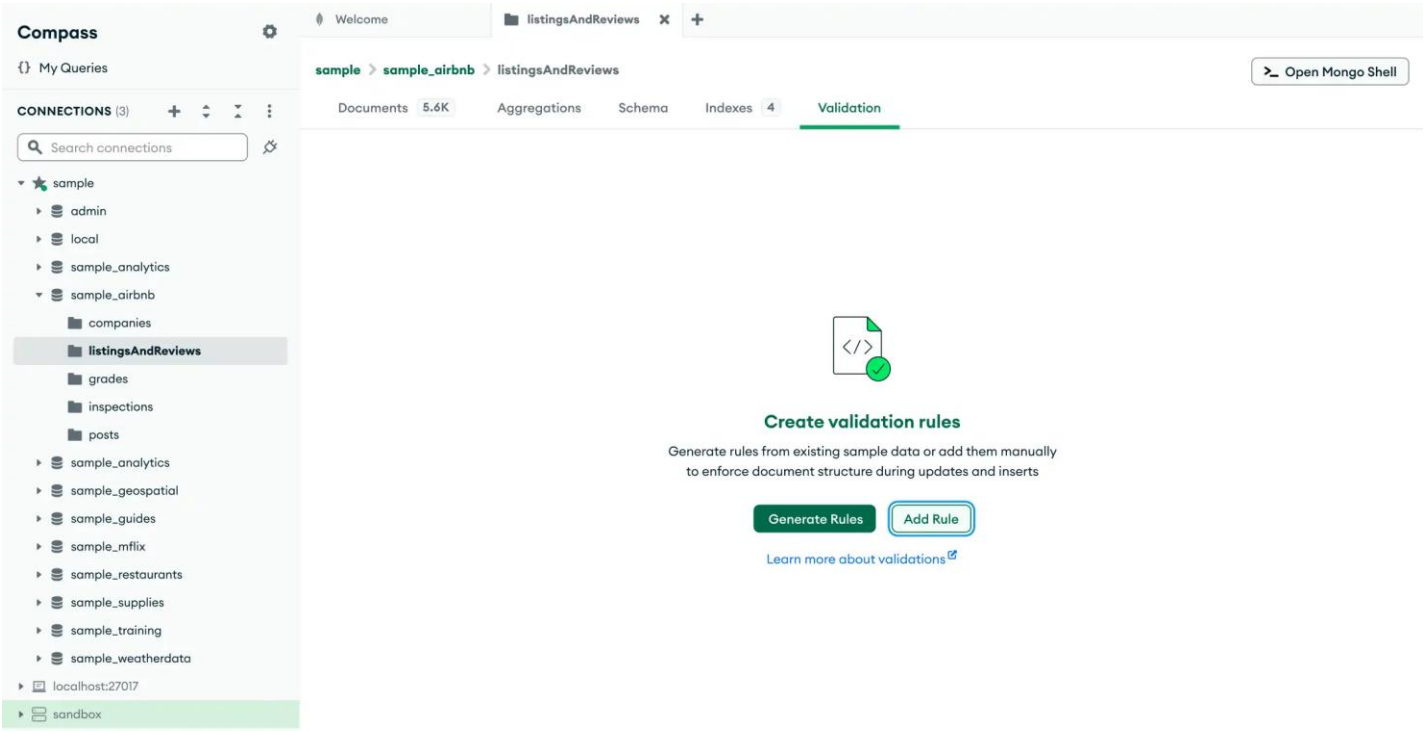
- You can also specify validation using query operators, with the exception of the following query operators: $near, $nearSphere, $text, and $where.
- Using this validation, one of the following must be true:
- The name field must be BSON type string.
- The borough field must be one of the enum strings.

```
{
   $or: [
      { name: { $type: "string" } },
      { borough: {
            bsonType: "string",
            enum: [ "Manhattan", "Brooklyn", "Queens", "Bronx", "Staten Island
            description: "must be one of the enum strings"
      } }
   ]
}
```

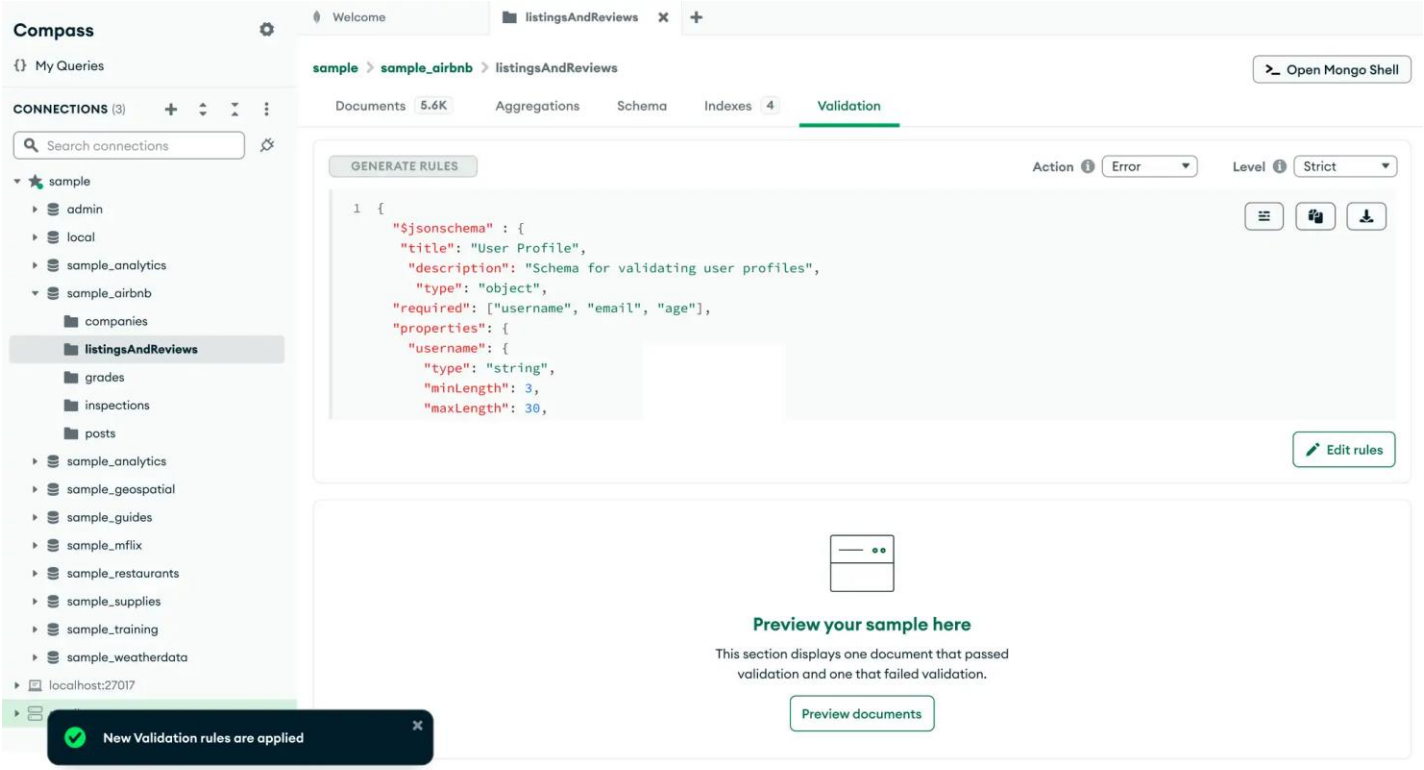
GENERATE VALIDATION RULES



ADD VALIDATION RULES



EDIT VALIDATION RULES



THANK YOU